Introduction to Ezairo® 7100 Programming

M-20797-010
September 2014

© SCILLC, 2014
Previous Edition © 2013
"All Rights Reserved"

ON Semiconductor®

Introduction to Ezairo 7100 Programming

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo and SignaKlara are either trademarks or registered trademarks of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction 5
   - 1.1 Purpose 5
   - 1.2 Intended Audience 5
   - 1.3 Conventions 5
   - 1.4 Manual Organization 6
   - 1.5 Further Reading 6

2. Architecture Overview 7
   - 2.1 Introduction 7
   - 2.2 CFX Digital Signal Processor 8
     - 2.2.1 CFX Memory. 9
   - 2.3 HEAR Configurable Accelerator 9
     - 2.3.1 Shared Memory 9
   - 2.4 ARM Cortex-M3 Processor 10
   - 2.5 Filter Engine 10
   - 2.6 FIFO Controller 11
   - 2.7 Input/Output Audio Control 11
   - 2.8 Input Stage 11
   - 2.9 Signal Detection Unit (SDU) 12
   - 2.10 Output Stage 12
   - 2.11 CFX Shared PCM and I2S Interface 12
   - 2.12 CFX Peripherals 12
   - 2.13 ARM Cortex-M3 Processor Peripherals 13
   - 2.14 CFX External Interfaces 14
   - 2.15 ARM Cortex-M3 Processor External Interfaces 15
   - 2.16 Clocking 15
   - 2.17 Power Management 15
   - 2.18 Firmware Components 16

3. Design Information 18
   - 3.1 Introduction 18
   - 3.2 Programming a Quad-Core System 18
     - 3.2.1 Partitioning Algorithms 19
     - 3.2.2 Other Design Considerations 21
   - 3.3 Data Flow 23

4. Introduction to Sample Code 26
   - 4.1 Introduction 26
   - 4.2 Accessing the Sample Code 26
   - 4.3 Accessing the Firmware Libraries 28
   - 4.4 Code Structure 28
     - 4.4.1 CFX Code Structure 28
     - 4.4.2 ARM Cortex-M3 Core Code Structure 30
   - 4.5 File Structure 31
     - 4.5.1 CFX Source Code File Structure 31
     - 4.5.2 HCT Input File Structure 31
     - 4.5.3 Filter Engine Source Code File Structure 31

4.5.4 ARM Cortex-M3 Core File Structure . . . 31
4.6 Application Initialization . . . 32
4.7 Modifying the Sample Code Applications—Examples . . . 33
4.7.1 Modifying the CFX Code . . . 33
4.7.2 Modifying the ARM Cortex-M3 Core Code . . . 37
5. Porting Guide . . . 38
5.1 Introduction . . . 38
5.2 Development Tools . . . 38
5.2.1 Integrated Development Environment . . . 38
5.2.2 CFX Toolchain . . . 39
5.2.2.1 Changes to CFCC Command Line Usage . . . 39
5.2.3 Changes to HCT Command Line Usage . . . 39
5.2.4 Migrating Ezairo 5900 or Ezairo 6200 Projects . . . 40
5.2.4.1 Creating a New CFX Project . . . 40
5.2.4.2 Copy Source Code Files . . . 40
5.2.4.3 Modify source code files for Ezairo 7100 . . . 41
5.2.5 Chip IDs . . . 41
5.3 Memory . . . 41
5.3.1 CFX Memory . . . 41
5.3.2 HEAR Shared Memory . . . 42
5.4 Analog Blocks . . . 42
5.5 Clocking . . . 43
5.6 FIFO Controller Input and Output Blocks . . . 43
5.6.1 FIFO and IOC Macros . . . 44
5.7 HEAR Library Migration . . . 44
5.8 Digital Input/Output (DIO) Interface . . . 44
5.8.1 DIO Macros . . . 44
5.8.2 GPIO Macros . . . 45
5.9 EEPROM Library . . . 45
5.10 Program ROM . . . 46
A. Glossary . . . 47

CHAPTER 1
Introduction

1.1 Purpose

The intention of this document is to help you start developing applications on Ezairo® 7100 as quickly as possible. Whether you are new to the DSP technology of ON Semiconductor or experienced, you can benefit from the general design information in this manual. If you have used Ezairo 5900 or Ezairo 6200, this manual will help you transition to Ezairo 7100 by describing changes to the hardware and firmware, and telling you how to migrate your projects. If you are a new user, you can benefit from the use of sample code to learn about writing programs for Ezairo 7100.

This manual describes some of the advancements in the chip over previous generations that, when properly exploited, can provide improved performance and open up possibilities for implementing new features in your algorithms. Key improvements include:

- More computing cycles with lower power consumption, allowing you to run more sophisticated speech processing algorithms
- An ARM® Cortex™-M3 processor that functions as a co-processor in the system to provide fast interrupt response features, increase the total computation capacity, and facilitate efficient handling of peripherals
- A wireless data handler block that can emulate industry standard codecs such as CSVD and G.722, and is flexible enough to interface with proprietary and future wireless chips
- A signal detection unit
- A Filter Engine to perform basic filtering and arithmetic operations on audio signals
- A low delay path that (with signal processing blocks on the path) allows hearing aid performance algorithms such as Active Noise Cancellation (ANC) and Occlusion Management
- More on-chip memory
- More FIFO registers
- Support for digital microphone input and output
- More Digital Inputs/Outputs (DIOs) which allow hearing aid manufacturers to add new peripherals
- Larger addressing modes to communicate with larger external EEPROM
- Source code compatibility with Ezairo 5900 and Ezairo 6200: the CFX core and HEAR Configurable Accelerator are still part of the new chip.

This manual merely introduces you to programming Ezairo 7100. For details, see the other manuals listed in Section 1.5, “Further Reading” on page 6.

1.2 Intended Audience

This manual is aimed at software development professionals or signal processing engineers who are new to Ezairo 7100, whether they have previous experience with Ezairo 5900 or Ezairo 6200 or not. We assume readers have experience developing applications in assembly language, and are somewhat familiar with digital signal processing, embedded software development, or both.

1.3 Conventions

This manual contains programming examples for the CFX DSP and ARM Cortex-M3 core, and source file examples for use with the HEAR and the HEAR Configuration Tool. All of these examples use a fixed-width font to differentiate them from the rest of this manual's content, as shown in the example below:

```
// This is an example of Ezairo 7100 assembly code.
load a0, 0x7fffff
```

Many programming examples make use of definitions and macros supplied by standard header files such as *sk4_hw.cfi*. For the sake of simplicity, the `#include` statements are not shown explicitly. Note that the definitions from the header files generally use all capital letters, making it easier to differentiate them from the assembly or C language instructions.

1.4 Manual Organization

Introduction to Ezairo 7100 Programming contains the following chapters and appendix:

- **Chapter 1: Introduction**, describes the purpose of this manual, outlines how the manual is organized, and gives a list of suggested manuals to read for more information.
- **Chapter 2: Architecture Overview**, provides a basic architectural overview of Ezairo 7100.
- **Chapter 3: Design Information**, provides general information to remember when programming for Ezairo 7100.
- **Chapter 4: Introduction to Sample Code**, describes how to access and use the sample code provided with Ezairo 7100.
- **Chapter 5: Porting Guide**, describes how to port applications based on Ezairo 5900 or Ezairo 6200 to Ezairo 7100. This includes changes to the hardware and firmware, and how to use the programming tools.
- **Appendix A: Glossary**, provides definitions for new terms and abbreviations that are introduced in this manual.

1.5 Further Reading

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide for Ezairo 7100*
- *Ezairo 7100 Installation Guide*
- *CFX DSP Architecture Manual*
- *Hardware Reference Manual for Ezairo 7100*
- *Firmware Reference Manual for Ezairo 7100*
- *HEAR Configurable Accelerator Reference Manual*
- *Filter Engine Reference Manual*
- *SK4 ELF Toolchain Reference for Ezairo 7100*

CHAPTER 2
Architecture Overview

2.1 INTRODUCTION

Ezairo 7100 is a complete system for developing advanced digital hearing aid products. It provides an audio signal processing platform with numerous programmable and configurable components. It also includes firmware that can be used to help develop a complete signal processing application.

The Ezairo 7100 system is an asymmetric quad-core architecture, mixed-signal system-on-chip designed specifically for audio processing. It centers around four processing cores: the CFX Digital Signal Processor (DSP), the HEAR Configurable Accelerator, the ARM Cortex-M3 processor, and the Filter Engine core.

- The CFX DSP core is used to configure the system and the other cores, and it coordinates the flow of signal progressing through the system. The CFX DSP can also be used for custom signal processing applications that cannot be handled by the HEAR or the Filter Engine.
- The HEAR is designed to perform both common signal processing and complex standard filterbank operations such as the WOLA filterbank, reducing the load on the CFX DSP Core.
- The ARM Cortex-M3 processor, which functions as a coprocessor in Ezairo 7100, offers fast interrupt response features, wireless capability, and additional computation.
- The Filter Engine provides basic filtering and arithmetic operations on audio signals, and is tightly integrated with the audio path to provide an ultra-low delay path.

The four cores have a number of local and shared memories available to them. The CFX DSP, the HEAR and the ARM Cortex-M3 cores process data brought into system memory by the Input/Output Controller (IOC). The IOC can handle inputs from either an analog input stage or pulse code modulation (PCM) interface input, and can handle outputs to a digital direct-drive output stage or PCM interface output. The FIFO controller simplifies access to data by the CFX DSP, the HEAR, ARM Cortex-M3 core and the IOC by providing configurable hardware-based first-in, first-out (FIFO) buffers.

A top-level block overview of the components of the Ezairo 7100 system is provided in Figure 1.

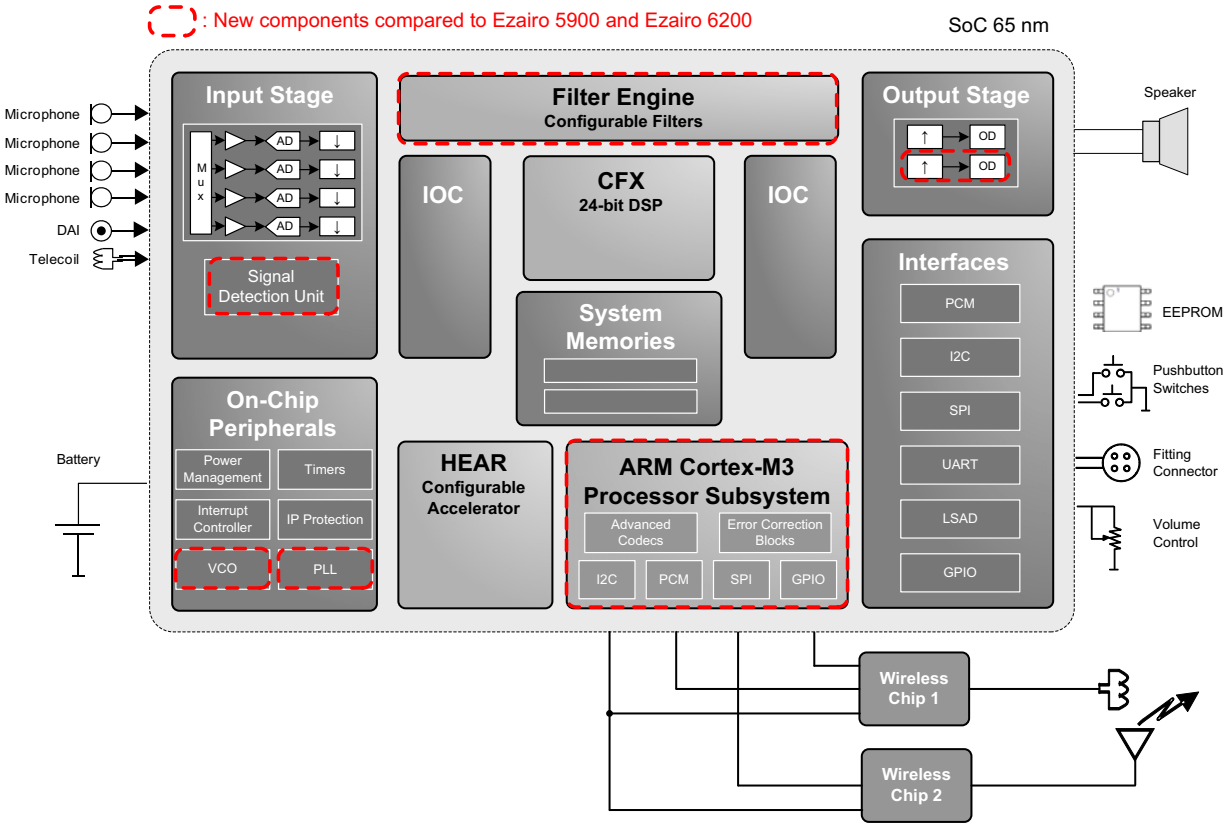


Figure 1. Top-Level System Block Diagram

Concise information about the various hardware components is provided in the subsections below to help you get started on programming Ezairo 7100 quickly.

For more detailed information about each hardware component, refer to the *Hardware Reference Manual for Ezairo 7100*. A brief introduction to the firmware components of Ezairo 7100 is included in Section 2.18, “Firmware Components” on page 16. For comprehensive information about the firmware components, refer to the *Firmware Reference Manual for Ezairo 7100*.

2.2 CFX Digital Signal Processor

The CFX DSP is the primary digital processor in the system. It is a user-programmable, general-purpose DSP. The CFX serves many roles in the system:

- Acts as the primary controller in the system
- Interacts closely with its peripherals and external interfaces
- Configures and controls the other cores and ARM Cortex-M3 core’s peripherals
- Used to coordinate and assist in data and control processing tasks

The CFX DSP is closely tied to its local memories, but also has access to memories of the HEAR Accelerator, the Filter Engine, and the ARM Cortex-M3 core.

More information about the CFX DSP is available in the *CFX DSP Architecture Manual* and the *Hardware Reference Manual for Ezairo 7100*.

2.2.1 CFX Memory

The CFX DSP uses a dual-Harvard memory bus architecture to provide separate buses for the program memory and two data memories.

The program memory space available to the CFX includes a program ROM as well as program RAM space to store a user application and interrupt vectors. This memory is accessible as instructions to the CFX through the instruction memory bus and as data for the CFX through the P memory bus. In addition, the P memory space provides access for the CFX to the HEAR core's microcode memory, and to the system control and configuration registers for all four cores.

The data memory areas available to the CFX are divided into two memory spaces that may be accessed either separately through the X memory and Y memory buses as 24-bit data, or concatenated together as 48-bit XY memory data. The X memory bus also provides access for the CFX DSP to the HEAR shared memory and configuration registers.

2.3 HEAR Configurable Accelerator

The HEAR Configurable Accelerator is a 24-bit signal processing engine that provides standard and specialized signal processing functions. It provides essential signal processing functionality such as FIR/IIR filtering, vector-based arithmetic operations, and WOLA filterbank functions. It is highly configurable at the hardware and microcode levels. The microcode for the HEAR is organized into modules which perform a single data processing task through one or more microcode functions. You can organize these functions into groups called function chains which are then triggered by hardware events such as a command from the CFX, or an interrupt from the FIFO controller. The CFX can start, pause, or reset the HEAR as needed.

The HEAR has access to shared memory for storage of inputs, outputs, coefficients and intermediate results. The HEAR also has an integrated Block Floating Point (BFP) unit, which is used when accessing the shared memory. The BFP unit automatically detects the magnitude of the data being written, and rescales the data during the next read to avoid overflow. The HEAR has dedicated microcode memory for storing the application's microcode configuration.

The execution of function chains on the HEAR is managed by the function chain controller. The HEAR Configurable Accelerator supports 24 different function chains—16 can be triggered by commands from the CFX and 8 can be triggered by interrupts from the FIFO controller (with configurable sources from any of the 32 FIFOs). The CFX can configure the function chain controller to control which function chains are active as well as the priority of the different function chains. The CFX can receive up to eight interrupt signals from the HEAR to indicate the completion of individual microcode functions or entire function chains. As part of the microcode configuration process, you can configure the specific events to trigger the function chains, as well as the timing of the interrupt signals that the HEAR sends back to the CFX.

For more information about the HEAR Configurable Accelerator and its associated hardware, consult the *Hardware Reference Manual for Ezairo 7100*. For more information about microcode configuration and available functions, consult the *HEAR Configurable Accelerator Reference Manual*.

2.3.1 Shared Memory

The shared memory provides data storage for the HEAR Configurable Accelerator. It is also accessible from the CFX and acts as the primary means of interfacing between the two cores. All HEAR shared memory is 48-bits wide and the CFX can address individual 24-bit words within each memory. All memories are single-ported and the shared memory bus controller controls access to them.

Shared memory is composed of the following distinct physical memories:

- H0, H1, H2, H3, H4 and H5 memories are all 128x48-bit memories generally used for filterbank processing such as analysis results, gain values and scratch memory.
- A0, A1, B0 and B1 memory are 1024x48-bit memories which are connected to the FIFO controller, and hence are generally used for FIFOs.
- C0, C1, D0 and D1 memory are 1024x48-bit memories generally used for coefficients as well as microcode configuration data and microcode module state variables.

The memories described above can also be accessed in an interleaved fashion. This interleaving is done at the 48-bit word level (i.e., alternating 48-bit words come from two different physical memories). This allows the HEAR to access data 96 bits at a time and in a manner that is advantageous for certain algorithms, such as accessing samples and coefficients in an interleaved fashion.

For more information about the shared memory, consult the *Hardware Reference Manual for Ezairo 7100*.

2.4 ARM Cortex-M3 Processor

The ARM Cortex-M3 processor is a low-power processor intended for deeply embedded applications that require fast interrupt response features.

The ARM Cortex-M3 processor is slave to the CFX which has master control of the Ezairo 7100 system. The inter-processor communication methods between the ARM Cortex-M3 processor and the rest of Ezairo 7100 are:

Shared memory	The CFX can access any memory or peripheral in the ARM Cortex-M3 processor's memory space except for the internal peripherals mapped onto the ARM Cortex-M3 core's private peripheral bus.
HEAR FIFO access	The FIFO in the HEAR subsystem is mapped to a portion of the ARM Cortex-M3 processor data memory space.
Interrupts	The ARM Cortex-M3 processor has 8 input interrupts from the CFX. In addition, the ARM Cortex-M3 processor can generate up to four interrupts to the CFX.

For more information about the ARM Cortex-M3 processor, consult the *Hardware Reference Manual for Ezairo 7100*.

2.5 Filter Engine

The Filter Engine is a specialized core that provides low-delay paths and basic filtering capabilities for the Ezairo 7100 system. It can implement filters (either FIR or IIR) with a total of up to 160 coefficients.

Common use cases for the Filter Engine include low-delay paths, multi-rate signal processing, and pre- and post-filtering for the audio signal. However, in general, the Filter Engine can share the load with other cores by taking any signal processing task that is suitable for its architecture.

The Filter Engine must be configured by the CFX, which also controls its starting and pausing. The Filter Engine is programmable, but does not include direct debugging access. The CFX can monitor the Filter Engine state through control and configuration registers on the IOMEM bus. There are also limited debug assistance features, such as manual execution and single step execution. For more information about the Filter Engine, consult the *Filter Engine Reference Manual* and the *Hardware Reference Manual for Ezairo 7100*.

2.6 FIFO Controller

The FIFO controller controls access to the A0, A1, B0 and B1 shared memories, allowing portions of this memory to act as hardware managed FIFO buffers. This controller serves three purposes in managing access to shared memory:

- It re-maps portions of the address space so that a hardware circular buffer is implemented, and the most recent samples in the buffer can always be accessed at the same logical address.
- It resolves access conflicts between the input/output controller (IOC) and the three system cores that access these memory areas through the shared memory bus.
- It tracks incoming data to a buffer or outgoing data from a buffer. Changes to these buffers can trigger function chains on the HEAR, interrupts on the CFX, or interrupts on the ARM Cortex-M3 core.

More information about configuring FIFOs and using the FIFO controller is available in the *Hardware Reference Manual for Ezairo 7100*.

2.7 Input/Output Audio Control

Input/Output audio control on Ezairo 7100 consists of the Input/Output Controller, and audio multiplexing with the Filter Engine.

The Input/Output Controller (IOC) manages the transfer of audio data into and out of the system through the input stage, output stage and PCM interface. The IOC can be configured to acquire input data from any combination of the four input channels (which can select among the six analog inputs and four digital microphone signals) and the two PCM input channels. The IOC is also used to direct output data to any combination of the two sigma delta modulators (which can output to the direct digital output drivers, analog outputs, or digital microphone outputs), and the two output PCM channels. The IOC routes the data to and from the HEAR shared memory via the FIFO controller. This allows you to organize the IOC input and output data into various combinations of single-channel and multi-channel FIFO buffers.

With audio multiplexing, the Filter Engine is linked to the audio input and output stage to optionally pre-process the input signal and post-process the output signal. It also provides processing for the low-delay path.

More information about the configuration and operation of the IOC and audio multiplexing is available in the *Hardware Reference Manual for Ezairo 7100*.

2.8 Input Stage

The input stage of the Ezairo 7100 system is composed of four configurable audio input channels, which provide 18-bit digital samples to the rest of the system via the IOC. Each input channel includes:

- Input selection using an analog multiplexer to select between 4 input sources for each channel from the available 6 analog input sources
- Digital multiplexer to select between analog input channels and digital microphone sources
- Programmable input preamplification to adjust the selected raw input signal to a suitable level for the A/D converters
- An anti-aliasing low-pass filter with a selectable 20kHz or 50kHz (bypass) -3dB cut-off frequency
- Available line-out for the preamplified and filtered input signal
- An over-sampling analog-to-digital converter (ADC) which uses a programmable sampling frequency and provides a configurable sampling delay
- High-quality decimation filtering at all selectable sampling rates
- Selectable input muting
- Configurable power consumption for the ADC components

More information about the configuration and usage of the input stage is available in the *Hardware Reference Manual for Ezairo 7100*.

2.9 Signal Detection Unit (SDU)

The signal detection unit (SDU) is used to detect the presence of a signal on one of the analog inputs. The SDU consists of the following components:

- An input selection block to select which analog input channel is monitored
- A flexible preamplifier, similar to that used for the regular audio input channels
- A bypass for the preamplifier
- A 12-bit analog-to-digital converter (ADC)
- A circular buffer to hold 8 samples

More information about the SDU is available in the *Hardware Reference Manual for Ezairo 7100.*

2.10 Output Stage

The output stage of the Ezairo 7100 system is composed of two audio output channels that post-process data from the rest of the Ezairo 7100 system and provide it to an external receiver, speaker or digital microphone input interface. Each output channel includes:

- High-quality interpolation filtering that automatically tracks the selected sample rate of the sigma-delta modulator
- Ultra low-power, high fidelity, over-sampled sigma-delta D/A conversion with programmable sample rates
- A direct digital output driver for zero-bias hearing aid receivers and standard loudspeakers with a selectable high-power mode that combines multiple output pads for maximum output signal power
- A configurable low-pass filtered analog output suitable for connection with other audio devices
- Two digital microphone outputs

More information about the configuration and usage of the output stage is available in the *Hardware Reference Manual for Ezairo 7100*.

2.11 CFX Shared PCM and I^2S Interface

The Ezairo 7100 system has a configurable pulse code modulation (PCM) interface which can also operate in an Inter-IC Sound (I^2S) compatible mode. The PCM interface is highly configurable, allowing it to interface with a wide variety of external devices. In particular, it supports a configuration that is compatible with digital audio streaming from a NOAHlink device.

The PCM interface provides two additional digital inputs and two additional digital outputs for audio data. The IOC treats these inputs and outputs in a similar manner to the input and output stage signals.

For more information about the configuration and usage of the PCM and I^2S interface and the NOAHlink digital audio streaming mode, refer to the *Hardware Reference Manual for Ezairo 7100*.

2.12 CFX Peripherals

The Ezairo 7100 system provides a number of peripherals that are controlled through the CFX DSP core. The CFX peripherals include:

Interrupt Controller

The interrupt controller manages the signalling of interrupts from external interfaces and peripherals to the CFX DSP. Interrupt vectors are stored in P memory at a fixed location.

General-Purpose Timers

Two general-purpose timers are available in the Ezairo 7100 system. Each timer can be configured for a range of frequencies and granularities. Each timer can also operate in a free-run or one-shot mode.

Watchdog Timer

The watchdog timer is used to monitor the application's execution. Two successive watchdog timeouts triggers an automatic reset of the system.

Cyclic Redundancy Code (CRC) Generator

A 16-bit CRC generator is included to help with data integrity testing. The system firmware extensively uses the CRC generator but it can also be useful in end-user applications.

Access Mode Security Blocks

The Ezairo 7100 system includes a restricted access mode which can be used to secure intellectual property in the form of algorithms or data stored in memory. Data in non-volatile memory can also be stored in an encrypted format and decoded during system startup.

More information about configuring and using the CFX peripherals is available in the *Hardware Reference Manual for Ezairo 7100*.

2.13 ARM Cortex-M3 Processor Peripherals

The Ezairo 7100 system provides additional peripherals tightly integrated with and controlled through the ARM Cortex-M3 processor. These peripherals include:

Nested Vectored Interrupt Controller (NVIC)

The ARM Cortex-M3 processor is closely tied to a nested vectored interrupt controller (NVIC) which is an integral part of the processor and provides the interrupt handling functionality. The NVIC handles a non-maskable interrupt (NMI), several faults, pre-defined interrupts and 42 external interrupts.

SysTick

The SysTick count-down timer is available as part of the NVIC.

Error Correction and Coding Block

The error correction and coding block is a memory-mapped peripheral that contains the following four hardware accelerators: Hamming encoder, Hamming decoder and interleaver, Reed-Solomon encoder and Reed-Solomon decoder.

Audio Coding Block

The audio coding block is a memory-mapped peripheral that contains two hardware audio codecs: the G.722 encoder/decoder, and the CVSD encoder/decoder.

Direct Memory Access (DMA) Controller

The direct memory access (DMA) controller module allows background transfers between peripherals and memory without core intervention. The DMA is connected to the ARM Cortex-M3 core, the peripherals, and the SRAM via four independent channels.

Cyclic Redundancy Check (CRC) Generator

A hardware CRC generator that supports both the CRC-CCITT and CRC-32 (IEEE 802.3) algorithms.

More information about configuring and using the ARM Cortex-M3 processor peripherals is available in the *Hardware Reference Manual for Ezairo 7100*.

2.14 CFX External Interfaces

The Ezairo 7100 system provides a number of external interfaces that can be controlled through the CFX DSP. These interfaces provide support for different types of external communications, as well as signaling through general-purpose input/output (GPIO) pads and low-speed analog to digital (LSAD) converters. The external interfaces provided include:

Inter-IC Communications (I^2C) Interface

The Ezairo 7100 system contains three I^2C interfaces that share the same I^2C bus within the device.The CFX debug port I^2C interface is a hardware debugger interface tightly coupled with the CFX DSP and its memory components. The ARM Cortex-M3 processor debug port I^2C interface provides debug access to all registers of the ARM Cortex-M3 processor and all memory components attached to this core. A general-purpose I^2C interface is available for using the Ezairo 7100 system as an I^2C master or slave. This interface can operate at speeds of up to 400 kbps.

Digital Input/Output (DIO) Interface

The Ezairo 7100 system's DIO interface is an extension of the GPIO interface for Ezairo 5900 and Ezairo 6200. Thirty DIO pads can be configured for a variety of digital output modes, for a general-purpose input mode, or as an LSAD input.

General-Purpose Input/Output (GPIO) Interface

Ezairo 7100 can configure any of the DIO pads as CFX software-controlled general-purpose digital I/O (GPIO) pads.

Low-Speed Analog-to-Digital (LSAD) Interface

The LSAD interface provides a low-frequency, 10-bit interface for sampling slow-varying signals. The LSAD interface can be connected to six external and two internal signals for tasks such as battery monitoring.

Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) of the Ezairo 7100 system is designed to communicate with external devices with a particular focus on EEPROM and related non-volatile memory (NVM) devices. The system firmware, which provides EEPROM access, uses this interface extensively.

Universal Asynchronous Receiver-Transmitter (UART) Interface

A general-purpose two-pin UART interface is available for RS-232 compatible communications. The baud rate (bits/second) of this interface is typically configurable within a range of 0.4 to 320 kbps, depending on the application's system clock.

Note that some physical pins used for these interfaces are multiplexed, so not all interface capabilities are available simultaneously in an application. More information about configuring and using the CFX external interfaces is available in the *Hardware Reference Manual for Ezairo 7100*.

2.15 ARM Cortex-M3 Processor External Interfaces

The Ezairo 7100 system provides a number of external interfaces which can be controlled through the ARM Cortex-M3 core, including:

Inter-IC Communications (I^2C) Interface

The ARM Cortex-M3 core external interfaces includes an I^2C interface that supports both master and slave mode transfers.

General-Purpose Input/Output (GPIO) Interface

Ezairo 7100 can configure any of the DIO pads as ARM Cortex-M3 processor software-controlled general-purpose digital I/O (GPIO) pads.

Serial Peripheral Interface (SPI)

The ARM Cortex-M3 processor SPI can be configured to operate as an SPI master device or an SPI slave device.

Pulse Code Modulation (PCM) Interface

The ARM Cortex-M3 processor SPI interface can be configured to operate in either full-duplex mode or half-duplex mode, at a communication speed up to the system clock frequency.

Data transfers using the I^2C, SPI and PCM interfaces can be controlled directly by the ARM Cortex-M3 processor or indirectly using the DMA.

More information about configuring and using the ARM Cortex-M3 processor external interfaces is available in the *Hardware Reference Manual for Ezairo 7100*.

2.16 Clocking

The Ezairo 7100 system uses a system clock (SYSCLK) as the source of all its clock domains. The system clock can be derived from the output of the internal voltage-controlled oscillator (VCO), phase-locked loop (PLL), or an external clock source.

A number of internal clocks are derived from the main system clock to operate the input stage, output stage, peripherals and external interfaces. More information about these clock domains and their configuration is available in the *Hardware Reference Manual for Ezairo 7100*.

2.17 Power Management

The Ezairo 7100 system is powered from a single battery voltage and generates a number of regulated supply voltages for different system components. These regulated voltages provide low-noise power to the analog components of the system and external devices such as a microphone, and provide high current load power for external EEPROM

and RF chips. The system also includes integrated power management and battery monitor components to ensure proper system operation under varying power supply conditions.

More information about power supply issues and the power management components is available in the *Hardware Reference Manual for Ezairo 7100*.

2.18 Firmware Components

The Ezairo 7100 firmware is supplied as a set of include files and libraries which are accessible using the ON Semiconductor Ezairo 7100 IDE. The libraries are included as source code, compiled object code, and in some cases, as ROM-based routines. The firmware is used to help build applications for the Ezairo 7100 device. It has the following major components:

CFX System Include Files and Macros

The CFX system include files provide preprocessor definitions for all major hardware components. These definitions provide labels and names for addresses of digital and analog registers, locations and sizes of bitfields and standard settings for bitfields. You can use these labels and names when configuring and accessing the various hardware components from a CFX DSP program. The system include files also provide preprocessor macros which further simplify the use of each hardware component. Using the standard system include files and macros increases the readability of your application code and can simplify porting applications to future versions of Ezairo 7100.

ARM Cortex-M3 Core Include Files and Macros

The ARM Cortex-M3 core include files provide preprocessor definitions for the ARM Cortex-M3 core hardware components. They include interrupt vector table descriptions, and labels and names for things like addresses and bit descriptions of control and configuration registers in the ARM Cortex-M3 processor memory map. These files conform to the Cortex Microcontroller Software Interface Standard (CMSIS).

CFX System Library

The CFX system library provides several useful functions that can be called from your application. The system library is used for tasks such as resetting the system or changing the operating clock frequency.

CFX Math Library

The CFX math library provides a set of optimized math functions which are useful in developing audio signal processing applications. These functions include basic mathematical operations such as log(x), e^x, square root, sine, cosine, reciprocal and division. The available functions also include signal processing operations such as attack/release filtering and various conversions between root-mean-square, power and decibel values. Some functions work with both scalar and vector inputs. These functions are helpful when developing the audio signal processing portion of your application.

ARM Cortex-M3 Processor Math Library

The ARM Cortex-M3 processor math library contains a set of fixed-point and floating-point functions that can be used for common time-domain data processing such as exponential averaging, attack-release averaging, and linear interpolation.

EEPROM Library

The EEPROM library provides a set of standard functions for communicating with an EEPROM connected to the Ezairo 7100 device's SPI port. This library provides high-level functions for configuring the EEPROM and for reading or writing data in different word and block sizes. The EEPROM library is useful for developing portions of an application that require access to non-volatile memory storage.

EEPROM File System

A standard EEPROM-based file system structure is used for Ezairo 7100 applications. The file system structure is supported by the Program ROM and the standard Bootloader firmware. It provides a structured framework for storing system data, application code and data tables. Portions of the file system can be encrypted using the AES encryption algorithm to provide protection of intellectual property in the form of application code or data.

Program ROM

The Program ROM is responsible for the booting and initialization of the Ezairo 7100 system. It also includes ROM-based versions of many of the other firmware libraries. The use of ROM-based libraries helps to reduce the program memory and EEPROM space required for your application.

Bootloader

A standard bootloader is provided with the Ezairo 7100 firmware. This bootloader provides support for loading and starting a user application stored in EEPROM using the Ezairo 7100 file system.

More information about the Ezairo 7100 firmware is available in the *Firmware Reference Manual for Ezairo 7100*.

CHAPTER 3
Design Information

3.1 Introduction

For all new users of Ezairo 7100, the following sections provide design recommendations to make efficient use of the chip's resources from a programmer's perspective. If you have used Ezairo 5900 or Ezairo 6200 before, you are already familiar with load-balancing the CFX and the HEAR, and can focus more on the Filter Engine and ARM Cortex-M3 core programming aspects.

Compared with Ezairo 5900 and Ezairo 6200, Ezairo 7100 has many improvements besides the addition of the Filter Engine and ARM Cortex-M3 cores. See Chapter 2, "Architecture Overview" on page 7 for an overview of Ezairo 7100 architecture.

3.2 Programming a Quad-Core System

Optimum power efficiency is achieved by using all cores equally, or load-balancing. The amount of power consumed is primarily determined by the core that does most of the work, so if that work is evenly divided among all cores, you can reduce the overall clock frequency, consuming less power. Figure 2 is an illustration of this concept for a simple case with a dual-core system.

Since the four cores have their own strengths besides sharing certain common capabilities, you can take apart your algorithm and assign different portions of it to run on different cores (called *algorithm partitioning*). If your algorithm requires a low-delay path, use the Filter Engine to construct this low-delay path first. If you need wireless communication, use the ARM Cortex-M3 core to enable the wireless audio functionalities. It is also generally desirable to use the ARM Cortex-M3 core to handle tasks not essential to the audio processing, so that you can leave the CFX and the HEAR to focus on the heavy processing that they are best at. We recommend that the vectorized, number-crunching portions run on the HEAR, and the rest of the signal processing tasks run on the CFX. Algorithm partitioning is key to optimizing the power efficiency of programs running on Ezairo 7100.

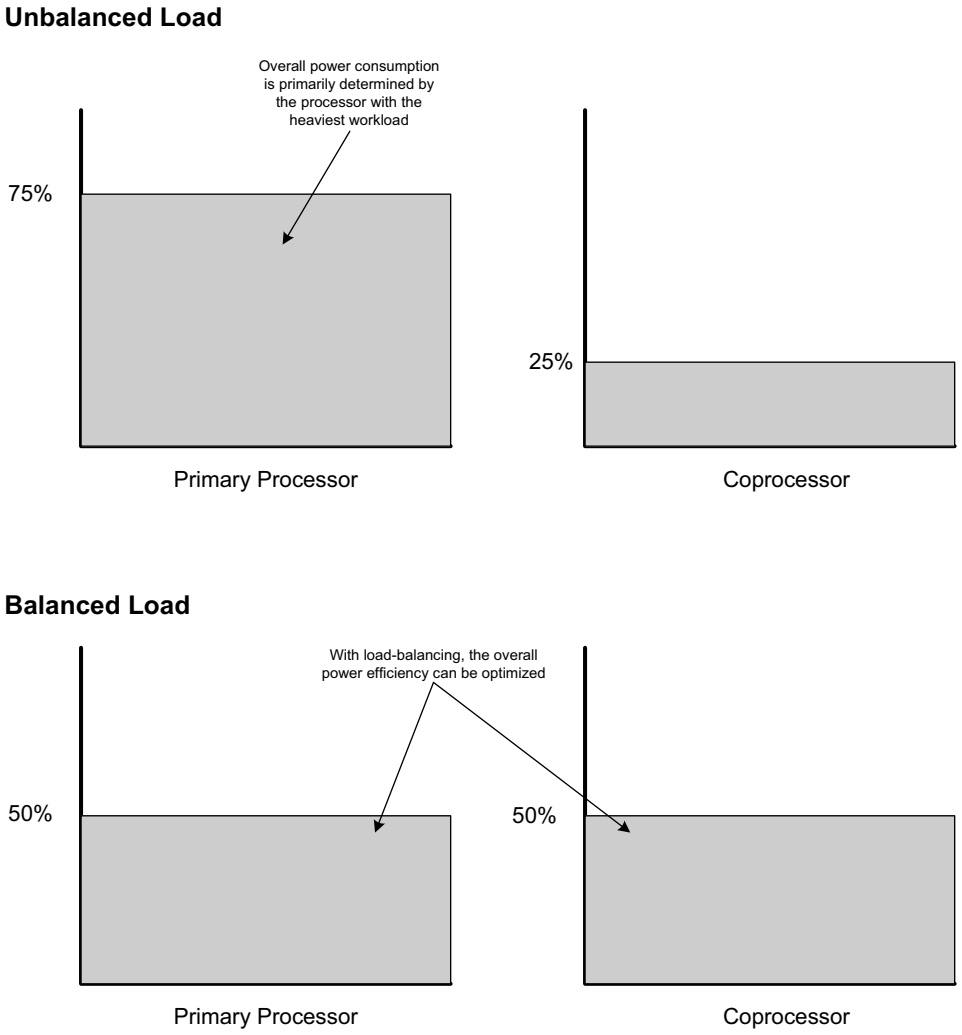


Figure 2. A Conceptual Illustration of Load-Balancing

3.2.1 Partitioning Algorithms

Partitioning algorithms for Ezairo 7100 is somewhat related to task scheduling for multiprocessor systems, a topic that has been studied extensively in the field of multiprocessor computing. This section describes a few key concepts that are useful to developers working on partitioning algorithms for Ezairo 7100.

The guidelines for partitioning algorithms for Ezairo 7100 are:

- Make each core do what it is best at and what no other core can do.
- Schedule tasks among the four cores so that the workload is distributed among them as evenly as possible.

Following these two guidelines, when analyzing an algorithm for partitioning, it is important to consider the algorithm from two perspectives: functional partitions and timing partitions.

Functional partitions Divide the algorithm based on the type of operations (or “functions”) that need to be run

Timing partitions Divide the algorithm based on when a part of the algorithm needs to be run, and how often and how quickly it needs to be completed.

When analyzing your algorithm from the functional perspective, keep in mind that:

- Sometimes certain tasks must be assigned to a particular core because either this core is the only one to perform them, or no other core would be nearly as efficient even if possible. For example, if your algorithm requires a low-delay path, you must configure the Filter Engine to establish it. If you need to use a CVSD codec, the ARM Cortex-M3 core hardware-based CVSD codec would be the natural choice.
- The HEAR is typically more power efficient than the CFX in running the functions provided by the microcode modules. If the cycle count for a given task is about the same on both cores, the HEAR is still likely to be more power efficient. Thus it is usually better to partition your algorithm in such a way that you can utilize the functions available from the HEAR and not duplicate the same functions on the CFX. See the *HEAR Configurable Accelerator Reference Manual* for the complete description of the available microcode modules.
- The communication scheme between the CFX, the IOC and the HEAR is presented in Table 1. You can start a function chain from a CFX command or a FIFO interrupt, and you can configure each interrupt from the HEAR to the CFX at various points within a function chain. For example, if a function chain contains an FIR filter function followed by a filterbank analysis function, you can configure the HEAR to raise an interrupt on the CFX after the FIR filter function has completed, and raise another interrupt after the filterbank analysis function has completed.

Table 1. HEAR Inter-Processor Communication

From	To	Number of Interrupts
HEAR	CFX	8
CFX	HEAR	16
FIFO	HEAR	8

- The communication scheme between the CFX, the IOC and the ARM Cortex-M3 core is presented in Table 2. Some examples of interrupt uses are:
  - You can start an audio encoding process on the ARM Cortex-M3 core as soon as an input FIFO is full.
  - The ARM Cortex-M3 core can raise an interrupt on the CFX when a particular function has completed.

Table 2. ARM Cortex-M3 Core Inter-Processor Communication

From	To	Number of Interrupts
ARM Cortex-M3 Core	CFX	16
CFX	ARM Cortex-M3 Core	8
FIFO	ARM Cortex-M3 Core	8

- The CFX and the ARM Cortex-M3 core should handle initialization and control of all peripherals and external interfaces on Ezairo 7100. The CFX can also configure and control most of the ARM Cortex-M3 core peripherals and external interfaces.

Furthermore, when analyzing your algorithm from the timing perspective, keep in mind that:

- For algorithms that require a low-delay path (LDP), ensure that a new output sample in the LDP is available at its sampling rate to maintain the continuity of the audio output. For example, if the system clock, and thus the Filter Engine clock, is at 2.56 MHz and the sampling rate of the LDP is 128 kHz, then you have a maximum of 20 cycles to complete any processing for each new sample. Typically, the CFX processing is not a part of the LPD. If you must use the CFX in the LDP, use caution and ensure that the timing constraint of the LDP is met.
- For algorithms that operate at the decimated sampling rate, ensure that a new block of audio data is available at the block rate to maintain the continuity of the audio output. Thus, the critical path of your task schedule is typically constrained by the block rate. The block rate is calculated as the sampling frequency divided by the input block size. For example, a 16 kHz sampling frequency divided by a block size of eight samples equals 2 kHz, meaning that a new block of audio data must be available every 0.5 ms.
- *Time-slicing* is a technique for partitioning your algorithm over the time dimension, to free up clock cycles for other more critical computations. It involves spreading parts of your algorithms over several blocks instead of having all the computations done within one block. For example, in a dynamic range compression algorithm, instead of calculating new compression gains for all 16 frequency bands within one block, time-slicing divides up the algorithm so that only the compression gains for bands 1-4 are calculated in the first block, then only the gains for bands 5-8 are calculated in the next block, and so forth. This way, the algorithm is partitioned so that it only needs to calculate the compression gains for 4 frequency bands at a time. Implementing time-slicing might compromise the performance of your algorithm if it is not implemented in a way that accounts for the different sample rates needed.

  Note that the multi-rate signal processing capability of the Filter Engine offers convenient time-slicing on the Filter Engine. For tasks that run at a decimated rate (as compared to the highest sampling rate for the “root” program sector), you can spread them across multiple program phases. Consult the *Filter Engine Reference Manual* for the concept of multi-rate processing, program phase and program sector.

- Either a command from the CFX or an event from the FIFO controller can launch a function chain on the HEAR. Thus it is important to consider the flow of the data in your algorithm to see if the FIFO controller should directly launch a function chain on the HEAR, while the CFX is running other computations.
- For those tasks that run on the ARM Cortex-M3 core, we recommend taking advantage of any available DMA channels for data transfers. If all four DMA channels have been used, you can reconfigure some or all of them at run-time. This might be power-efficient if the program overhead associated with DMA run-time reconfiguration is less than the computation cycles that manual data transfers would cost.

3.2.2 Other Design Considerations

Designing an audio processing system usually involves trade-offs between power consumption, circuit size and noise, as illustrated in Figure 3. A lower noise level implies a higher RMS current and more filtering, typically on the power supply, which requires a bigger size. Analog systems displayed similar behavior in that a bigger transistor with more current produced less noise.

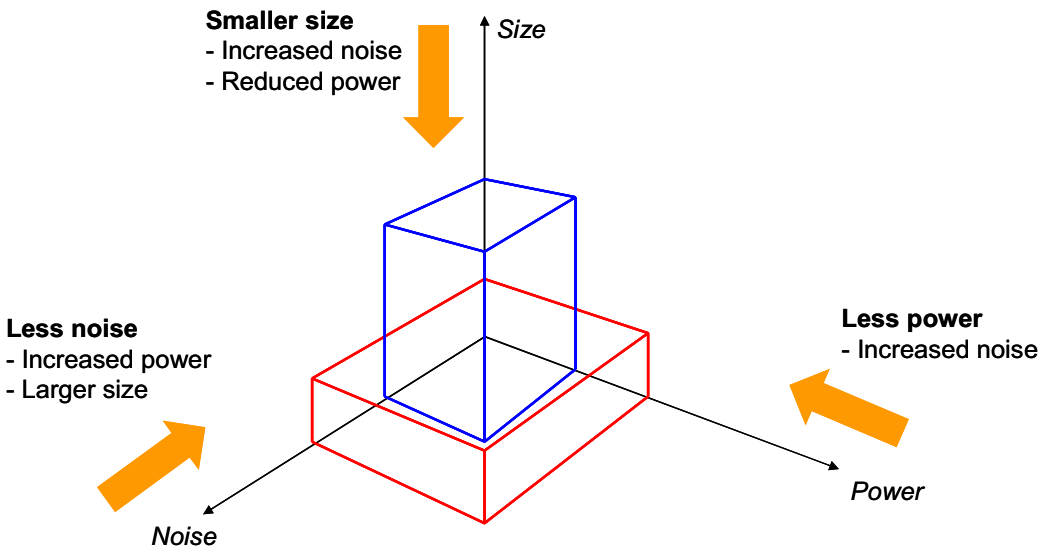


Figure 3. Trade-offs Between Power, Size and Noise when Designing an Audio System

In an audio processing system that emphasizes low power consumption and small circuit size, one way in which noise could be induced is if the dynamic current drawn by a processor varies periodically, and if the peak-to-peak level of that dynamic current is large enough. A tonal noise can be induced in the output signal because of current coupling with the rest of the system. For instance, for an unbalanced algorithm on Ezairo 5900 or Ezairo 6200, the HEAR might periodically draw a dynamic current because it first runs the filterbank analysis function, then idles when the CFX processes the analysis results. After it finishes processing, the HEAR runs more filterbank functions before idling again, and so on. If the HEAR and CFX processors periodically operate and idle, the peak-to-peak level of the dynamic current drawn by the processors might be very large, which can induce a tonal noise in the output signal. This is known as the “block rate tone” problem.

Ezairo 7100 has much improved power management schemes to mitigate this problem, but its complete elimination still requires careful load-balancing from your algorithm. On Ezairo 5900 and Ezairo 6200 you are encouraged to evenly distribute the workload between the CFX and the HEAR (load-balancing). Similarly, the flexibility of the Ezairo 7100 system allows you to implement efficient algorithm partitioning to evenly distribute the workload among all four processors, so that each processor is not periodically operating and idling. This means that the peak-to-peak level of the dynamic current will be very small, and no tonal noise is induced. Figure 4 illustrates the concept of implementing load balancing to reduce the periodic peak-to-peak variations in the dynamic current.

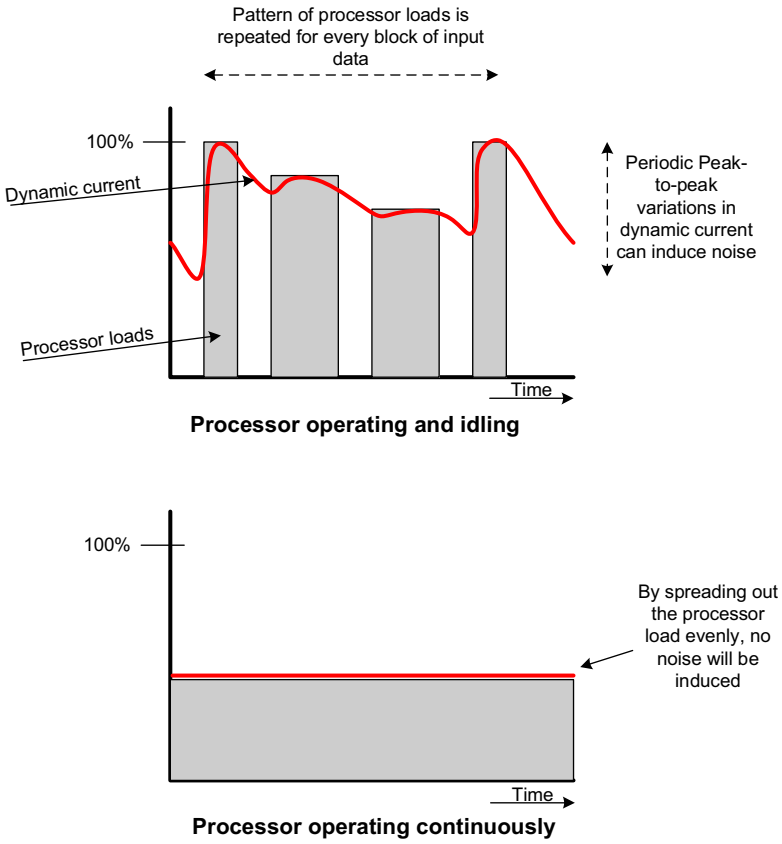


Figure 4. A Conceptual Illustration of Producing Noise and Avoiding It

Implementing load-balancing on Ezairo 7100 allows you to optimize the trade-off between power consumption and noise. It not only leads to lower dynamic power consumption, but it also causes lower average power draw, while at the same time avoids inducing tonal noise. By reducing the peak-to-peak variations that require more milli-Watts when a processor is operating in short bursts, you achieve the same results with continuous operation of the processors. In effect, the same amount of work is done when all active processors are running continuously, but with lower overall energy consumption. On Ezairo 7100, with the flexibility of the design that allows you to implement load-balancing effectively and improved power management schemes, you can avoid tonal noise.

Note that load-balancing does not mean you have to use all four cores. If a core is not in use and disabled, it does not consume power and does not induce noise.

IMPORTANT: If you do not implement proper load-balancing, your program running on Ezairo 7100 can induce tonal noise.

3.3 Data Flow

Ezairo 7100 features flexible data flow through the Input/Output Controller (IOC), the FIFO controller, audio multiplexing with the Filter Engine, and the DMA controller on the ARM Cortex-M3 core.

The IOC can be configured to automatically route data between the system memory on one side, and the input/output stage and PCM interface on the other side. The FIFO controller supports up to 32 FIFOs that can be configured to different sizes and locations within the system memory. If you are familiar with Ezairo 5900 or Ezairo 6200, you will notice the increased number of FIFOs on Ezairo 7100. Each of the FIFOs can be connected to one or more data sources or sinks. A data source is simply something that writes data into the FIFO, and a data sink reads data from the FIFO. For example, the input stage is a data source for an input FIFO, whereas a PCM port can both receive and transmit data, so it can be both a data source and a data sink. Moreover, a data source or sink is not required to be a physical interface connected to the IOC. An FIR filter, for example, running on the HEAR can be a data sink that reads from an input FIFO, while at the same time it can be a data source for another FIFO. FIFOs that are connected to a physical interface through the IOC are called *hardware FIFOs*. FIFOs that are controlled by the software on the CFX and HEAR are called *software FIFOs*. The IOC, CFX, HEAR and ARM Cortex-M3 core all have access to the FIFOs via the FIFO controller, which maintains a fixed access priority scheme. See the *Hardware Reference Manual for Ezairo 7100* for details about the IOC and FIFO controller, and also Section 5.6, “FIFO Controller Input and Output Blocks” on page 43.

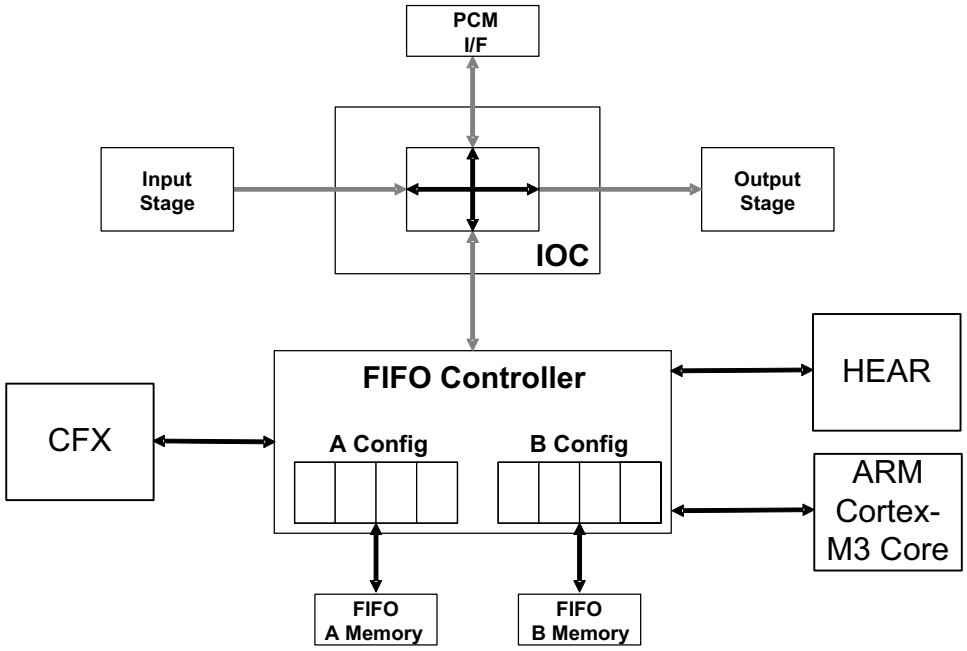


Figure 5. Flexible Data Flow via the IOC and FIFO Controller

To design your algorithms to take advantage of the flexibility provided by the IOC and FIFO controller, consider all the components that can be data sources, data sinks, or both. Configure the IOC and the FIFO controller such that they connect the data sources and sinks to the FIFOs that you have specified. Access to the FIFOs from within your program is straightforward—the FIFO controller handles all the pointer updates to the FIFOs, and automatically maps direct FIFO access to the appropriate location in the physical memory, so that your program does not need to calculate new addresses due to data updates in the FIFOs. For example, an input FIFO can be configured in such a way that the most recent block of data from the input stage is always located at the base of the FIFO. Thus, an FIR filter that processes data from the input FIFO can simply start from the same memory address every time a new block of data is available in the FIFO.

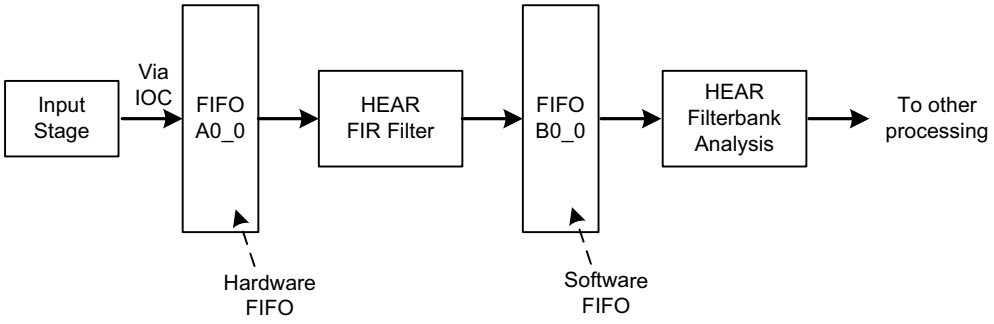


Figure 6. Comparing Hardware FIFOs to Software FIFOs

With audio multiplexing, you can route audio data through the Filter Engine for filtering and basic arithmetic operations. Typical examples of Filter Engine configuration include:

- Low-delay path
- Audio pre-processing on the input
- Audio post-processing on the output

See the *Hardware Reference Manual for Ezairo 7100* and *Filter Engine Reference Manual* for details about audio multiplexing.

The DMA controller on the ARM Cortex-M3 processor has flexible addressing modes to enable you to efficiently transfer data between ARM Cortex-M3 processor data memory (including the mapped HEAR FIFO memory) and peripherals. The DMA controller supports up to four independent channels and step sizes between -4 and 4. Each channel can be connected to one data source and one data sink. A DMA data source or sink can be a data buffer in the ARM Cortex-M3 processor memory map, or a peripheral such as the G.722 encoder or the SPI. See the *Hardware Reference Manual for Ezairo 7100* for more information about the DMA controller.

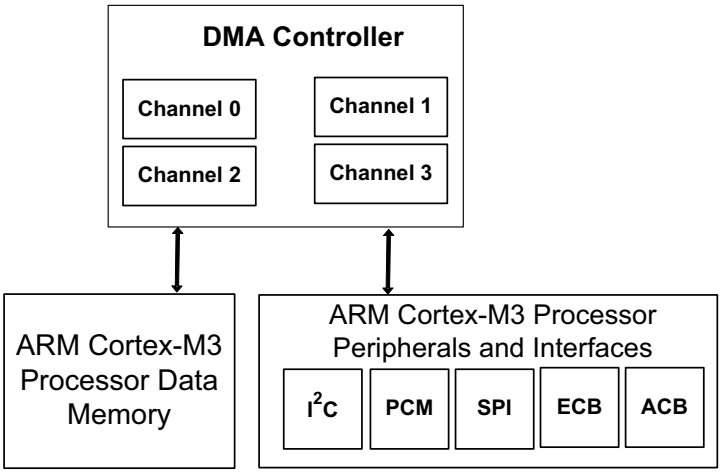


Figure 7. Flexible Data Flow Through the DMA Controller

CHAPTER 4
Introduction to Sample Code

4.1 Introduction

To help you learn about the various programming aspects of Ezairo 7100, the Integrated Development Environment (IDE) includes a number of sample code applications as part of its installation. Each sample code application is intended to be simple, easy to use and modify, while at the same time, is also practical and applicable. You might want to experiment with programming Ezairo 7100 by building an application based on a sample code template. Nevertheless, while the sample code does contain recommended programming practices for Ezairo 7100, it is primarily intended to be a learning tool and does not represent the most efficient or optimal way to implement programs for Ezairo 7100.

NOTE: Additional sample code is available in the Communication Toolkit (CTK) Developer Kit. It includes sample code applications for communicating with a host system such as a PC.

4.2 Accessing the Sample Code

The sample code applications are included in the IDE installation. To view the source code outside of the IDE, look in the directories listed in Table 3.

Alternatively, the sample code can be viewed through the IDE. To modify, build and run a particular sample, follow the steps in Method #1 or Method #2.

If you want to import all of the sample applications into the IDE at once, and do not need the ability to build the applications, follow the steps in Method #3.

A text file called *readme.txt* accompanies each sample code application. This file contains information about how that specific sample works.

NOTE: The default installation folder is specified in the *Ezairo 7100 Installation Guide*. Your installation folder is represented as **<EDK_HOME>** in this document.

Table 3. Installed Location of Sample Applications

Sample Application	Location of Compiled Application and Source Code
All-Core Passthrough	**<EDK_HOME>***source**samples**All_Core_Passthrough*
Battery Monitor	**<EDK_HOME>***source**samples**battery_monitor*
Calibration	**<EDK_HOME>***source**samples**Calibration_Example*
Data Logging	**<EDK_HOME>***source**samples**data_logging*
Digital Volume Control	**<EDK_HOME>***source**samples**Digital_Volume_Control*
Filter Engine Low-Delay Path	**<EDK_HOME>***source**samples**feng_low_delay*
Input Dynamic Range Extension	**<EDK_HOME>***source**samples**idrx*
Mono FIFO Copy	**<EDK_HOME>***source**samples**mono_fifo_copy*
Mono Filterbank Passthrough	**<EDK_HOME>***source**samples**mono_filterbank*

Table 3. Installed Location of Sample Applications (Continued)

Sample Application	Location of Compiled Application and Source Code
Multi-Filterbank Passthrough (Mixed)	**<EDK_HOME>***source**samples**multi_filterbanks_mixed*
Multi-Filterbank Passthrough (Mono)	**<EDK_HOME>***source**samples**multi_filterbanks_mono*
Filter Engine Multi-Rate Processing	**<EDK_HOME>***source**samples**multi_rate_feng*
PCM Passthrough	**<EDK_HOME>***source**samples**pcm_passthrough*
Sampling Delay	**<EDK_HOME>***source**samples**sampling_delay*
Simple-Stereo Filterbank Passthrough	**<EDK_HOME>***source**samples**simple_stereo_filterbank*
Single-Channel WDRC	**<EDK_HOME>***source**samples**single_channel_wdrc*
Soft Power Down	**<EDK_HOME>***source**samples**soft_power_down*
Stereo Filterbank Passthrough	**<EDK_HOME>***source**samples**stereo_filterbank*
Subband FIR/IIR Filtering on the CFX	**<EDK_HOME>***source**samples**subband_filter_cfx*
Subband FIR/IIR Filtering on the HEAR	**<EDK_HOME>***source**samples**subband_filter_hear*
Time-Domain FIR/IIR Filtering on the CFX	**<EDK_HOME>***source**samples**td_filter_cfx*
Time-Domain FIR/IIR Filtering on the HEAR	**<EDK_HOME>***source**samples**td_filter_hear*
Tone Generation	**<EDK_HOME>***source**samples**tone_generation*
Two-Channel WDRC	**<EDK_HOME>***source**samples**two_channel_wdrc*
Volume Control Application	**<EDK_HOME>***source**samples**volume*
Wireless Audio Passthrough	**<EDK_HOME>***source**samples**Wireless_Audio_Passthrough*

Method #1: Creating a copy of a single sample code application in the workspace

1. In the C/C++ perspective of the IDE, select from the menu **File > New > Project...**, or right-click in the empty space in **Project Explore** and select **New > Project...**.
2. Select **SK4 -> SK4 Project**, and click **Next**.
3. Type in a project name without spaces, and from the list **SK4 Executable / EEPROM** in **Project type**, select the sample code that you want to view, and click **Finish**. The IDE creates a project in your workspace using the sample code template.

This method copies one sample code application at a time into your workspace. You can use this copy as a template for your own applications.

Method #2: Import a sample code application as an existing project

1. From the menu, select **File > Import**.
2. Select **General > Existing Projects into Workspace** and click **Next**.

3. From **Select root directory**, browse to the location of the sample code applications in your file system. The location is **<EDK_HOME>***source**samples*\<*sample_name*>. Check **Copy projects into workspace**.Then click **Next**.

The selected sample code from the IDE installation is copied into your workspace. You can modify this to build your own application, just as in Method #1.

Method #3: Viewing multiple sample code applications through the IDE (no copies)

1. From the menu, select **File** > **New** > **Project**.
2. Select **General** > **Project** and click **Next**.
3. Type in a project name without spaces. You might want the project name to indicate that this project will contain multiple sample code projects.
4. Uncheck the **Use default location** check box and browse to the location of the sample code applications in your file system. The location is **<EDK_HOME>***source**samples*. Then click **Next**.
5. Under **Referenced projects**, leave all projects unchecked. Click **Finish**.

These projects have not been copied into your workspace; they are the original versions from the IDE installation. As such, they are read only. Within each sample code application there is a master directory that contains the executable for this sample code application. You can run and step through the sample code application on the Ezairo 7100 hardware.

4.3 Accessing the Firmware Libraries

The sample code applications use a number of macros from the firmware libraries, particularly in the initialization routine. Table 4 specifies where documentation and source code for these libraries can be found.

Table 4. More Information About Firmware Libraries

Macro Type	Documentation Source	Source Code Location
General Firmware Macros	*Firmware Reference Manual for Ezairo 7100*	**<EDK_HOME>***include**cfx* **<EDK_HOME>***include**cm3*
CFX Math Library Macros	*Firmware Reference Manual for Ezairo 7100*	**<EDK_HOME>***include**cfx**sk4_math.cfi* **<EDK_HOME>***source**cfx**mathlib*
CFX Calibration Library Functions	Firmware Reference Manual for Ezairo 7100	**<EDK_HOME>***include**cfx**sk4_calibrate.cfi* **<EDK_HOME>***source**cfx**calibrate*

4.4 Code Structure

The Ezairo 7100 system is very flexible, and thus, an application might be structured in many different ways. However, some application structures are more prone to errors than others. The program structure used in the sample code applications was selected because it is robust and easy to modify.

An Ezairo 7100 program has to include at least a CFX project. If the ARM Cortex-M3 core is used, the application must also include a ARM Cortex-M3 processor project. We will address the code structure for a CFX project and for an ARM Cortex-M3 processor project separately.

4.4.1 CFX Code Structure

The CFX project can optionally include HEAR microcode and Filter Engine program code.

The HEAR microcode is divided into function chains. Each function chain is associated with a particular command from the CFX or an event from the FIFO controller. The processing on the CFX is divided into a number of core blocks that occur relative to these function chains on the HEAR. The CFX transitions from one block or state to the next based on interrupts it receives from the HEAR or FIFO controller.

Once the Filter Engine is configured and enabled, it typically starts program code execution at the arrival of each new sample that is configured to trigger Filter Engine processing, and continues until the end of its program code without any command from the CFX. The exception to this is when the CFX is manually controlling the Filter Engine.

The CFX structure used by all sample code applications is shown in Figure 8.

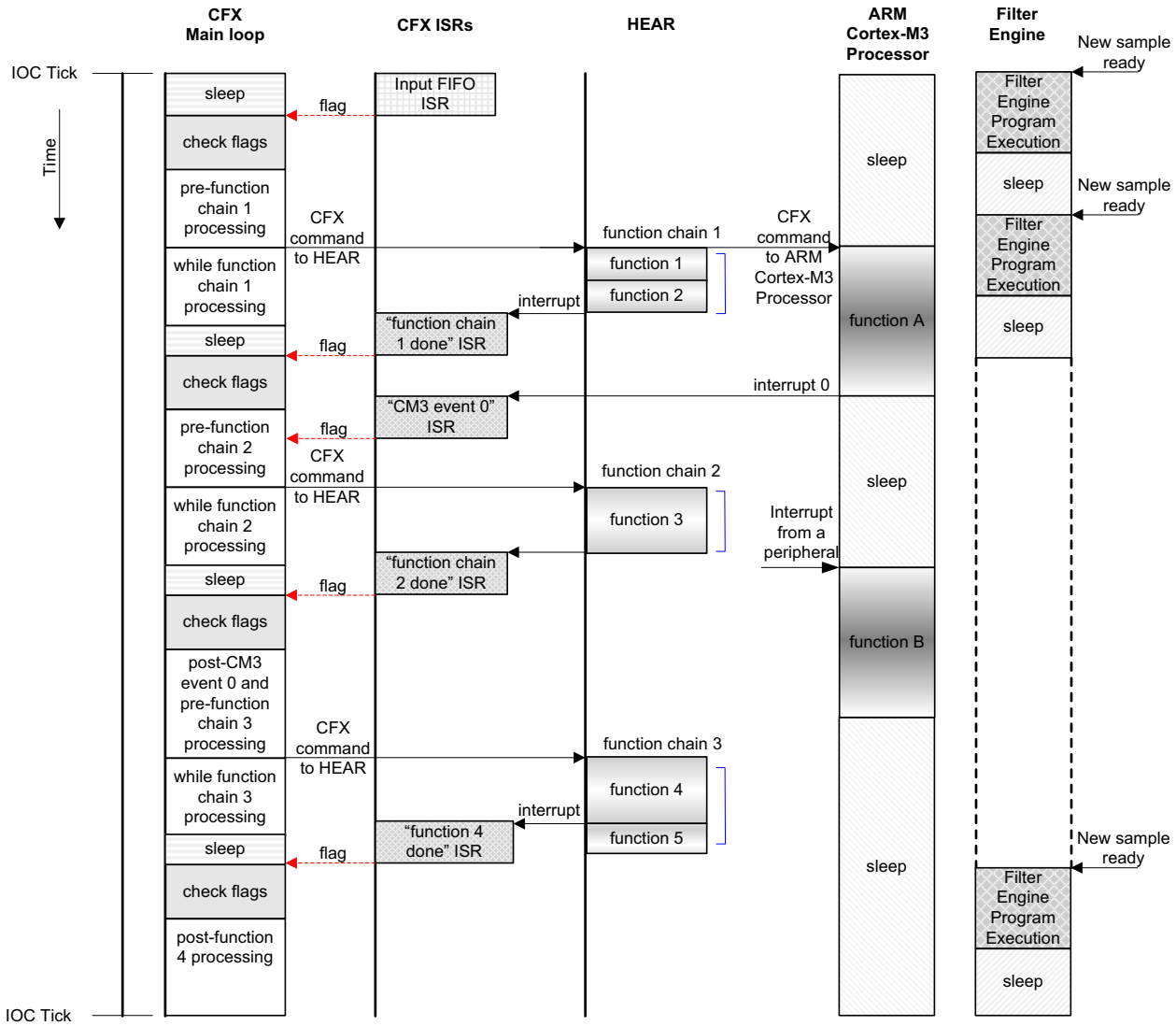


Figure 8. Sample Code Structure

Figure 8 illustrates how the processing on the CFX and the HEAR can be grouped into core blocks. These blocks occur relative to the function chains on the HEAR. The example above has three function chains. An interrupt is sent by the HEAR to the CFX at the end of function chains 1 and 2 and part-way through function chain 3 (at the end of function 4). Thus, the processing on the CFX and the HEAR can be divided into 7 distinct blocks:

- pre-function chain 1 processing
- while function chain 1 processing
- pre-function chain 2 processing
- while function chain 2 processing
- pre-function chain 3 processing
- while function chain 3 processing
- post-function 4 processing

These blocks are found in the *app.cfs* source file.

In each sample code application, the CFX launches the function chains from the main loop. When an interrupt is received by the CFX, a flag is set in the appropriate interrupt service routine (ISR). These ISRs are found in the *app_isr.cfs* source file. Each interrupt has a separate flag.

The main loop of each sample code application is structured as a simple state machine:

1. At the top of the loop, the program checks whether any interrupts have been received by checking the interrupt flags.
2. The flags are checked in the same order as the HEAR and FIFO interrupts are expected to arrive.
3. The code branches to the appropriate core block based on which flag has been set.
4. Once the core block has completed execution, the flags at the top of the main loop are checked again.
    - If no additional flags have been set, the CFX goes to sleep to save power.
    - If another flag has been set, the code branches to the appropriate core block.

The code for each core block is also found in the *app.cfs* source file.

For this state machine to work, command priorities on the HEAR and interrupt priorities on the CFX must be carefully considered. On both the CFX and the HEAR, there are two priority levels: normal and high. If all events have the same priority level, lower numbered events are treated with a higher priority level. For example, on the CFX, it is important that the HEAR 0 interrupt correspond to the first interrupt expected from the HEAR, the HEAR 1 interrupt correspond to the second interrupt expected from the HEAR, and so on. Similarly, it is important that command 0 correspond to the first function chain to run on the HEAR, command 1 correspond to the second function chain to run on the HEAR, and so on. For more information about interrupt priorities on the CFX and command priorities on the HEAR, refer to the *Hardware Reference Manual for Ezairo 7100*.

Figure 8 on page 29 also illustrates that the typical Filter Engine processing is independent from the CFX-HEAR processing. The Filter Engine starts processing its program code when a “sample ready” signal arrives, and continues until it reaches the end of its program. Note that the concept of an “IOC tick” does not apply to the Filter Engine. Instead, the Filter Engine must complete its processing and provide updated output samples within one sampling period of its “sample ready” signal.

4.4.2 ARM Cortex-M3 Core Code Structure

While we do not recommend a particular code structure for programming on the ARM Cortex-M3 core, we do recommend that you follow the Cortex Microcontroller Software Interface Standard (CMSIS).

The Ezairo 7100 IDE includes three sample code projects that use the ARM Cortex-M3 core: all-core audio passthrough, wireless audio passthrough, and digital volume control. They demonstrate using the ARM Cortex-M3 core to handle the peripherals and external interfaces, and using the DMA controller for transferring data efficiently.

4.5 File Structure

All sample code applications have a common source file structure. The files that are common to all sample code applications are described below.

4.5.1 CFX Source Code File Structure

The files that are common to the CFX portion of all sample code applications are described in Table 5.

Table 5. Common CFX Files in Sample Applications

File Name	Description
app.cfs	This file is the main source file for the CFX software. It contains the main loop of the application and the core blocks. This file can also contain application-specific functions such as filters.
app.cfi	This file is the include file for *app.cfs*. It includes numerous defines for the hardware and software configuration. It also declares global variables and functions.
app_init.cfs	This file contains the source code for the application's hardware initialization function, named `Initialize`.
app_isr.cfs	This file contains the source code for all of the application's interrupt service routines. An interrupt service routine exists for each interrupt received by the CFX from the HEAR, the FIFO controller, peripherals or interfaces.
app_memory.cfs	This file contains all of the memory allocation for the application.

4.5.2 HCT Input File Structure

The input file to the HEAR Configuration Tool (HCT) is *microcode.hct*. This file is used to configure the microcode that runs on the HEAR. For more information about the HCT, see the HCT options in Section 5.2.3, "Changes to HCT Command Line Usage" on page 39, and refer to the *HEAR Configurable Accelerator Reference Manual*.

4.5.3 Filter Engine Source Code File Structure

The Filter Engine source code file is *feng.fs*. This file contains the program code that the Filter Engine executes when a "sample ready" signal or a manual execution command from the CFX triggers Filter Engine processing. For more information, refer to the *Hardware Reference Manual for Ezairo 7100* and the *Filter Engine Reference Manual*.

4.5.4 ARM Cortex-M3 Core File Structure

The files that are common to the ARM Cortex-M3 core portion of all sample code applications are described in Table 6.

Table 6. Common ARM Cortex-M3 Processor Files in Sample Applications

File Name	Description
main.c	This file is the main source file for the ARM Cortex-M3 core software.
main.h	This file is the include file for *main.c*. It includes defines for the hardware and software configuration. It also declares global variables and functions.
initialize.c	This file contains the source code for the application's initialization function.
stack.c	This file contains the memory allocation for the stack.

4.6 Application Initialization

Each sample code application has a CFX initialization routine that can be found in the *app_init.cfs* source file. This initialization routine does the following:

- Configures the analog blocks
- Configures the FIFOs, the Input/Output Controller (IOC) and audio multiplexing
- Configures the HEAR's function chain controller
- Configures the Filter Engine
- Configures the CFX's Interrupt Controller
- Other system configurations
- Starts the HEAR if necessary
- Starts the ARM Cortex-M3 core if necessary
- Starts the Filter Engine if necessary

The CFX initialization routine makes extensive use of the CFX firmware macros. Refer to Section 4.3, "Accessing the Firmware Libraries" on page 28 for more information about how to access these macros. The inputs to these macros are defines that are specified in the include file *app.cfi*.

The CFX initialization routine is called in the *app.cfs* source file at the very beginning of the program (which is indicated by the label `main`). Once initialization has completed, code execution continues to the `waitloop`, where it remains until an interrupt occurs to start the state machine.

For each sample code application that uses the ARM Cortex-M3 core, there is an ARM Cortex-M3 core initialization routine that can be found in the *initialize.c* source file. This initialization routine does the following:

- Configures the Nested Vectored Interrupt Controller (NVIC)
- Configures the DMA controller
- Configures any ARM Cortex-M3 core peripherals and external interfaces that are used by the application

The ARM Cortex-M3 core initialization routine makes extensive use of the ARM Cortex-M3 core firmware macros. Refer to Section 4.3, "Accessing the Firmware Libraries" on page 28 for more information about how to access these macros. The inputs to these macros are defines that are specified in the include file *main.h*.

The ARM Cortex-M3 core initialization routine is called in the *main.c* source file at the very beginning of the program. Once initialization has completed, code execution continues to the main loop.

4.7 Modifying the Sample Code Applications—Examples

To modify a sample code application, the application must be imported into the IDE using Method #1, as described in Section 4.2, “Accessing the Sample Code” on page 26. In this section we discuss modifying the CFX part and the ARM Cortex-M3 core part of the application separately.

4.7.1 Modifying the CFX Code

One of the most challenging modifications that can be made to the CFX part of a sample code application is the addition of a function chain on the HEAR. This section describes the minimum steps that are involved in making this change to the structure.

Consider the following example. For simplicity we assume no change to the Filter Engine code and Filter Engine configuration. The sample code structure illustrated in Figure 8 on page 29 is to be modified through the addition of function chain 4. The new structure is shown in Figure 9.

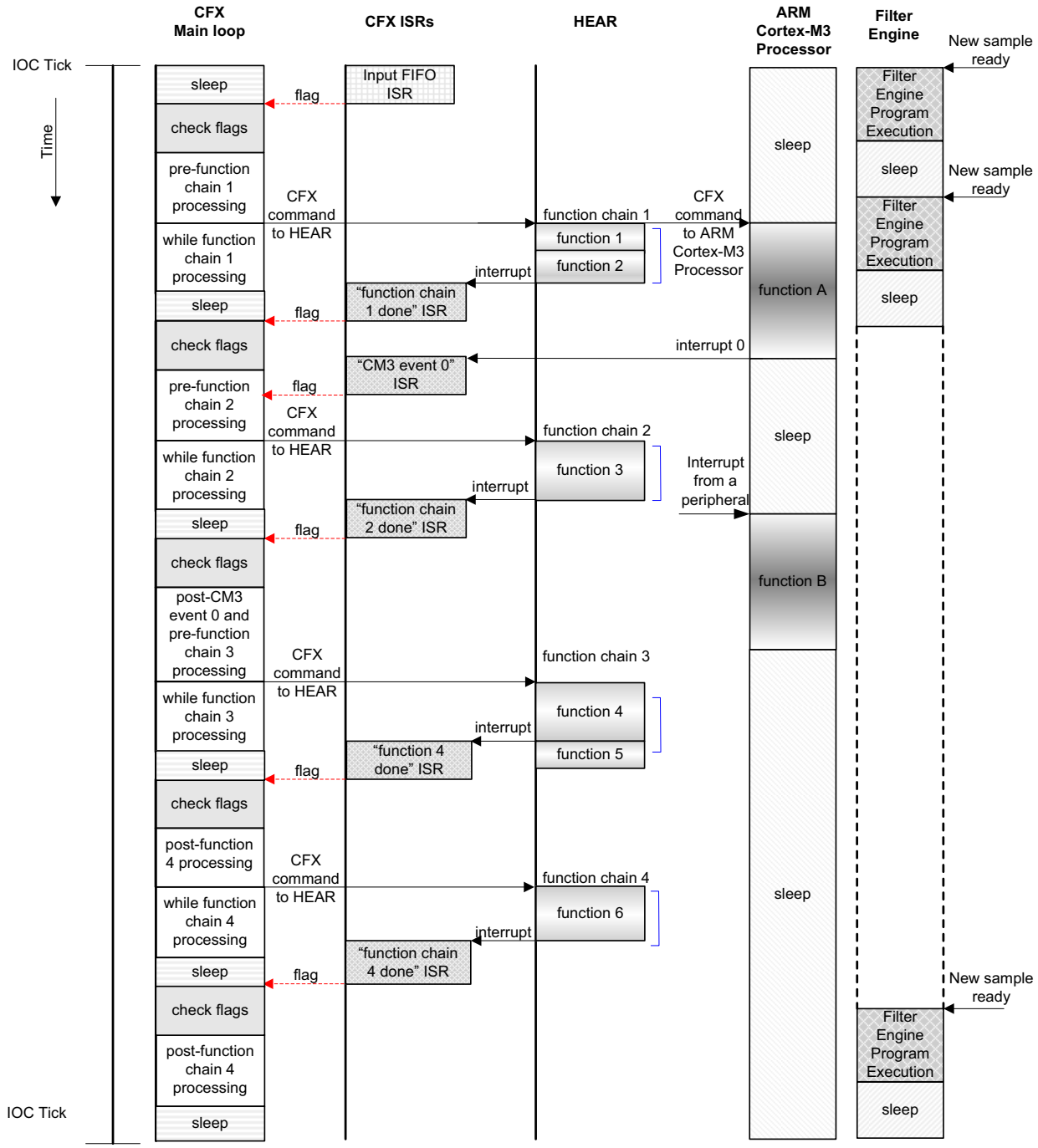


Figure 9. Modified Sample Code Structure

To add function chain 4 to the structure, follow these steps:

1. Add function chain 4 to the *microcode.hct* file:

```
//
// Function Chain 4
//
StartFunctionChain(CMD4, function_chain4)

        // Call some function (corresponding to function 6 in Figure 9)
        …

        // Trigger the HEAR 4 interrupt on the CFX
        Event(4)

EndFunctionChain()
```

In this example, command 4 from the CFX starts this function chain. The function chain triggers the HEAR 4 interrupt on the CFX upon completion.

The HCT uses this function chain definition to automatically update the define `HEAR_FC_ENABLE_VAL`, which can be found in *microcode.cfi*. This define specifies which function chains are enabled on the HEAR.

2. Add the HEAR 4 ISR to *app_isr.cfs*.

   Since function chain 4 triggers the HEAR 4 interrupt on the CFX, add an ISR for this interrupt. This ISR must set the appropriate flag to indicate that this interrupt has occurred. An example ISR is as follows:

```
// -------------------------------------------------------------------------
// Function:       HEAR_4_ISR
//  Description:         Sets a flag indicating that function chain 4 has completed
//                  on the HEAR
// Clock cycles:   3
// Modifies:       None
// -------------------------------------------------------------------------
HEAR_4_ISR:

    returni.2d

    // Set the HEAR function chain 4 done flag
    store x[X_HEAR_FUNCTION_CHAIN4_DONE], 1     // [ds1]
    nop                                         // [ds2]
```

3. Add `HEAR_4_ISR` to the interrupt vector table in *app_isr.cfs*.

   This entry in the interrupt vector table must change from:

```
goto Empty_ISR      // HEAR 4
```

   to:

```
goto HEAR_4_ISR     // HEAR 4
```

4. Allocate memory for the new flag `X_HEAR_FUNCTION_CHAIN4_DONE` in *app_memory.cfs*.

   Add the following to `X_System_Memory_Reserved` segment to allocate memory for `X_HEAR_FUNCTION_CHAIN4_DONE` and to initialize its value to 0:

```
// Initialize X_HEAR_FUNCTION_CHAIN4_DONE to 0
Sys_Mem_Reserve_Init(X_HEAR_FUNCTION_CHAIN4_DONE, 1, 0)
```

5. Specify X_HEAR_FUNCTION_CHAIN4_DONE as an externally defined symbol in *app.cfi*.

   Add the following line to *app.cfi* so that the flag X_HEAR_FUNCTION_CHAIN4_DONE can be referenced in all the files that include *app.cfi*:

```
.declare X_HEAR_FUNCTION_CHAIN4_DONE
```

6. Modify the INT_ENABLE_VAL define in *app.cfi* to enable the HEAR 4 interrupt on the CFX.

   Change the define INT_ENABLE_VAL from this:

```
#define INT_ENABLE_VAL       (INT_EBL_MASTER   | \
                              INT_EBL_WATCHDOG | \
                              INT_EBL_HEAR_0   | \
                              INT_EBL_HEAR_1   | \
                              INT_EBL_HEAR_2   | \
                              INT_EBL_HEAR_3   | \
                              INT_EBL_FIFO_0)
```

   to this:

```
#define INT_ENABLE_VAL       (INT_EBL_MASTER   | \
                              INT_EBL_WATCHDOG | \
                              INT_EBL_HEAR_0   | \
                              INT_EBL_HEAR_1   | \
                              INT_EBL_HEAR_2   | \
                              INT_EBL_HEAR_3   | \
                              INT_EBL_HEAR_4   | \
                              INT_EBL_FIFO_0)
```

7. Check the X_HEAR_FUNCTION_CHAIN4_DONE flag in the waitloop in *app.cfs*.

   Add the following code snippet below the code segment that checks and processes the X_HEAR_FUNCTION_CHAIN3_DONE flag:

```
// Check the HEAR Function Chain 4 done flag
load x0, x[X_HEAR_FUNCTION_CHAIN4_DONE]
cmp x0, 1 ||| nop
goto HEAR_function_chain4_done if eq
```

   This code snippet checks the X_HEAR_FUNCTION_CHAIN4_DONE flag and branches to the core block of code indicated by the label HEAR_function_chain4_done, if appropriate.

8. Add the “post-function chain 4 done” core block in *app.cfs*.

   The following is an example of a block of code that could be added:

```
HEAR_function_chain4_done:

    // Clear the flag
    store x[X_HEAR_FUNCTION_CHAIN4_DONE], 0

    // Re-enable interrupts
    Sys_Int_Master_Enable
    // Do whatever needs to be done at this point
    ...

    // Branch back to the top of the main loop
    goto waitloop
```

4.7.2 Modifying the ARM Cortex-M3 Core Code

Modifying the ARM Cortex-M3 core code is generally easier than modifying the CFX code because you program the ARM Cortex-M3 core in C. A potential challenge to modifying the ARM Cortex-M3 core code lies in adding a peripheral or external interface. In this case, first consult the *Hardware Reference Manual for Ezairo 7100* on how to control and configure this peripheral or interface.

Consider the following example of adding a DMA channel. Suppose DMA channels 0, 1 and 2 are already in use and you need to add DMA channel 3 for additional data transfers. Follow these steps:

1. Determine how your additional data transfer fits into the existing data path and if the source of your DMA channel 3 relies on the completion of any other task. For example, it might need a new block of data from a FIFO, or it might be directly connected to the destination of another DMA channel.
2. Make the necessary modifications to the existing application, such as adding a FIFO interrupt service routine in which you can start the DMA channel 3 transfer.
3. Add these defines in *main.h*.

```
/* DMA 3 configuration */
#define DMA_CH3_CFG_VAL                 ...
#define DMA_CH3_DEST_ADDRESS            ...
#define DMA_CH3_SRC_ADDRESS             ...
#define DMA_CH3_TRANSFER_LENGTH         ...
#define DMA_CH3_COUNTER_INT_VAL         ...
```

4. Configure DMA channel 3 in *initialize.c* using the defines created in the previous step.

```
/* Configure DMA channel 3 */
Sys_DMA_Channel3_Config(DMA_CH3_CFG_VAL,          \
                        DMA_CH3_TRANSFER_LENGTH,  \
                        DMA_CH3_COUNTER_INT_VAL,  \
                        DMA_CH3_SRC_ADDRESS,      \
                        DMA_CH3_DEST_ADDRESS);
```

5. Optional: if necessary, enable the DMA 3 interrupt in *initialize.c*.

```
/* Enable individual external interrupts */
NVIC_EnableIRQ(DMA3_IRQn);
```

6. Optional: if you enable DMA 3 interrupt from step 4, then add the DMA 3 interrupt service routine in *interrupts.c*.

```
/* ------------------------------------------------------------------------
 * Function      : void DMA3_IRQHandler(void)
 *  ----------------------------------------------------------------------------------------
 * Description   : Handler for the DMA 3 interrupt
 * Inputs        : None
 * Outputs       : None
 * Assumptions   : None
 *  ----------------------------------------------------------------------------------------*/
void DMA3_IRQHandler(void)
{
    /* To do when DMA 3 interrupt occurs */
}
```

CHAPTER 5
Porting Guide

5.1 INTRODUCTION

This chapter acts as a porting guide for developers familiar with Ezairo 5900 or Ezairo 6200 application development. It describes how to migrate an Ezairo 5900 or Ezairo 6200 to Ezairo 7100, and provides an overview of the hardware and firmware differences so that you can change your applications accordingly.

This chapter can also serve as a reference for important programming features of Ezairo 7100. As a reference, it can be useful to developers who are not familiar with Ezairo 5900 or Ezairo 6200.

We assume you are familiar with the ON Semiconductor development tools such as the Integrated Development Environment (IDE) and the appropriate assembly programming toolchains, or that you will find the necessary context in the appropriate manuals.

5.2 DEVELOPMENT TOOLS

Compared to the EDK support provided for Ezairo 5900 and Ezairo 6200, the Ezairo 7100 EDK has a new CFX toolchain based on the open source GNU Binutils toolchain.

The following functionality is still handled without user intervention:

- The Communication Toolkit (CTK) runtime correctly identifies an attached Ezairo 7100 according to its chip identifier (CID) and provides appropriate chip information, including registers and a memory map, to the debugger and other software tools.
- The CTK runtime uses an appropriate EEPROM helper binary to provide EEPROM transfers at a performance level comparable to Ezairo 5900 or Ezairo 6200.

5.2.1 Integrated Development Environment

The IDE used for Ezairo 7100 development is similar to that for Ezairo 5900 or Ezairo 6200 development, but is dedicated to Ezairo 7100 development.

To use the IDE for the first time, follow the steps below:

1. To start the IDE, go to the Windows Start menu, and select **ON Semiconductor** > **Ezairo 7100 IDE**.
2. When you open the IDE for the first time, you are prompted to select a workspace for the session. You can agree to use the suggested location, or type in a different location.

   The workspace is the work area for all your IDE projects. When you create a new project, it is created in a subfolder of this workspace folder (although you can select a different location when you create the project).

IMPORTANT: We recommend that you create a new workspace for Ezairo 7100 development rather than open an existing one that you created with another Eclipse-based product, such as the ON Semiconductor Ezairo 7100 IDE. Opening an existing workspace with the Ezairo 7100 IDE might cause the workspace to be modified in ways that are incompatible with the other product.

3. After selecting the workspace location, the IDE is launched with the Welcome screen. The Welcome screen contains links to introductory information.
4. When you are ready to start using the IDE, click the curved arrow that is labeled **Workbench** on the Welcome screen.

5.2.2 CFX Toolchain

The Ezairo 7100 EDK utilizes a new CFX toolchain that is based on the open source GNU Binutils toolchain. Until Ezairo 6200, EDK products offered by ON Semiconductor utilized tools based on a proprietary technology. The new toolchain has replaced the proprietary one as the default toolchain for Ezairo 7100 development. Because this new toolchain generates its output in the widely-used standard ELF file format, it is referred to here as the SK4 ELF toolchain.

See Table 7 for a list of file extension conventions for the SK4 ELF toolchain.

Table 7. SK4 ELF Toolchain File Extensions

File Type	ELF Toolchain
assembly code	*.cfs*
assembly header file	*.cfi*
preprocessed assembly code	*.cft*
object file	*.o*
library	*.a*
executable	*.cfe*

5.2.2.1 Changes to CFCC Command Line Usage

Table 8 summarizes the differences between previous Ezairo products and Ezairo 7100 in command line options.

Table 8. CFCC Options for Ezairo 7100 that Differ from Ezairo 5900 and Ezairo 6200

Option	Description
`-b` **`<machine>`**	Selects the target architecture. Specify `SK4` for **`<machine>`**.
`--toolchain=`**`<toolchain>`**	Selects the back end toolchain. Valid values for **`<toolchain>`** are `gnu` and `archelon` (default).

`-p`
`--lpl` **`<lines>`**
`--lb`
`--cfpack`
`--xref`
`--xrefopts` **`<opts>`**
`--dbgopts` **`<opts>`**
`--line` **`<n><,m...>`**
	Same as in Ezairo 5900 and Ezairo 6200 EDK. Not available with `--toolchain=gnu`.

5.2.3 Changes to HCT Command Line Usage

Table 9 summarizes the changes to HCT command line usage versus what was documented in the *HEAR Configurable Accelerator Reference Manual* of EDK 5.2.

Table 9. HCT Options for Ezairo 7100 that Differ from Ezairo 5900 and Ezairo 6200

Option	Description
`-b` **<machine>**	The argument **<machine>** specifies the target machine for compilation. It recognizes names in the form **<arch>**`c`**<cid>**, where **<arch>** is the instruction set architecture, and **<cid>** is the chip ID. For Ezairo 7100, specify `sk4c102`.
`--toolchain=`**<toolchain>**	Selects the back end toolchain. Use `gnu` for **<toolchain>**.

5.2.4 Migrating Ezairo 5900 or Ezairo 6200 Projects

You cannot automatically migrate Ezairo 5900 or Ezairo 6200 projects to Ezairo 7100 projects; however, you can migrate a project manually.

Start by creating a new CFX project in the IDE as described below. Copy all of your source files from the Ezairo 5900 or Ezairo 6200 project to the new one in the **Project Explorer** view, then edit your source code to reflect Ezairo 7100 hardware and firmware changes.

5.2.4.1 Creating a New CFX Project

This section describes how to create a new CFX project in the Ezairo 7100 IDE that is configured to build with the new ELF tools. Such tools use the C/C++ support provided by the Eclipse CDT project and rely on makefiles and the UNIX build utility, Make, to manage their build processes.

These projects will use cs-make, a Windows port of GNU Make included in the CodeSourcery ARM toolchain. To learn more about GNU make, consult the GNU make manual at:

http://www.gnu.org/software/make/manual/make.html.

Follow these steps to create a new empty project that uses the new ELF toolchain to build a CFX program:

1. Launch the Ezairo 7100 IDE as described in Section 5.2.1, “Integrated Development Environment” on page 38.
2. Launch the C/C++ Project Wizard by choosing **File > New > C Project** from the main menu.
3. Enter a **Project name**.
4. Under **Project type > SK4 Executable / EEPROM**, choose `Empty SK4 project`.
5. Click the **Finish** button to create the project.

5.2.4.2 Copy Source Code Files

After you create the new CFX project as explained above, copy your Ezairo 5900 or Ezairo 6200 project source code files using either one of the two methods below:

- In Windows Explorer, copy the Ezairo 5900 or Ezairo 6200 source code files to the new project folder. Notice that these new files are then reflected in the project explorer in the C/C++ perspective in the Ezairo 7100 IDE workspace.
- Drag the Ezairo 5900 or Ezairo 6200 source code files from Windows Explorer and drop them into the new project folder in the project explorer in the C/C++ perspective in the Ezairo 7100 IDE workspace.

NOTE: Only copy the source code files, not the project configuration files or any build artifacts.

5.2.4.3 Modify source code files for Ezairo 7100

After copying your Ezairo 5900 or Ezairo 6200 source code files to the new CFX project, change any *sk3_*.cfi* include files to *sk4_*.cfi* in the source code, then edit the source code to reflect Ezairo 7100 hardware and firmware changes.

Major hardware and firmware changes are explained in the rest of this chapter to give you an overview of the porting effort involved. However, you might also need to consult the relevant sections in the hardware and firmware reference manuals to get all the information you need to port specific code segments in your Ezairo 5900 or Ezairo 6200 project.

Another way to help you port your project is to compare a sample project under the Ezairo 5900 or Ezairo 6200 architecture with the same project under the Ezairo 7100 architecture. For example, if your Ezairo 5900 or Ezairo 6200 project is closely related to the mono filterbank sample project, and you are porting it to Ezairo 7100, you could use a text comparison software to compare the mono filterbank sample project source files under the old and new architectures. Once you identify and understand the code differences, you can modify your own project more easily.

5.2.5 Chip IDs

The chip ID (CID) for Ezairo 7100 is 102. See the *Firmware Reference Manual for Ezairo 7100* for more information about SK4 variants, and how you can determine your device's chip ID.

Projects that are being built to support Ezairo 7100 should specify a CID of 102 (chip version 1.02.xx) using the symbol `SK4_CID`. You can either set this symbol in the source files or through the project build settings (build properties or preprocessor settings). See the *SK4 ELF Toolchain Reference for Ezairo 7100* for more information about build settings.

5.3 MEMORY

5.3.1 CFX Memory

Ezairo 7100 uses the same memory architecture as Ezairo 5900 or Ezairo 6200, but includes many updated memory instances. Table 10 lists the memory structures attached to the CFX, the size and width of each memory structure, as well as the bus or buses that the CFX DSP uses to access the memory. For detailed information on the memory structure, see the Hardware Reference Manual for Ezairo 7100.

Table 10. CFX Memory Structures

Memory Structure	Data Width	Memory Length	CFX Access Using	Compared to Ezairo 5900	Compared to Ezairo 6200
Program memory (ROM)	32	12288	P memory bus	Increased	Increased
Program memory (RAM)	32	16384, configurable to 24576	P memory bus	Increased	Increased
Microcode memory (RAM)	32	4096	P memory bus	Increased	Same
X memory (RAM)	24	12288	X, XY memory buses	Increased	Increased
Math Library look up table (ROM)	24	128	X memory bus	Same	Same
Y memory (RAM)	24	4096	Y, XY memory buses	Increased	Increased

5.3.2 HEAR Shared Memory

Two differences from HEAR development on Ezairo 6200 or Ezairo 5900 are worth noting here:

- The CFX mapping of the HEAR shared memory is significantly different (see the *Hardware Reference Manual for Ezairo 7100* for details). This affects the usage of HEAR address parameters in both the HEAR configuration file and CFX source code files. If you are porting an existing HEAR configuration file or a CFX source code file with references to the HEAR shared memory, ensure that address parameters are specified using the new memory layout. We recommend that you always use the built-in symbolic names for the memory banks so that you automatically use the correct memory layout.
- The named memory banks in the HEAR (A, B, C, D) are each split in two. For example, Ezairo 7100 has A0 and A1 memories, whereas Ezairo 5900 and Ezairo 6200 have only A. Several address parameters in the Standard HEAR Library are restricted to a particular named memory bank. When you pass values for these parameters in an Ezairo 7100 HEAR configuration, the address you pass can fall in either the 0 or 1 part of the memory. For example, when the *HEAR Configurable Accelerator Reference Manual* documents that a parameter is restricted to A memory, you can pass an address that is in either A0 or A1.

The shared memory instances and their sizes (measured in 24-bit addressable words) are summarized in Table 11. Size comparisons to Ezairo 5900 and Ezairo 6200 is also listed. For detailed information about how these memories map to the CFX address map, see the Hardware Reference Manual for Ezairo 7100.

Table 11. HEAR Shared Memory Instances

Memory Structure	Memory Length	Size Compared to Ezairo 5900 or Ezairo 6200
H0 memory	512	Increased
H1 memory	512	Increased
H2 memory	512	Increased
H3 memory	512	Increased
H4 memory	512	Increased
H5 memory	512	Increased
SIN/COS Data Table (ROM)	2048	Increased
A0 memory	2048	Same size as A memory
A1 memory	2048	New; same size as A memory
B0 memory	2048	Same size as B memory
B1 memory	2048	New; same size as B memory
C0 memory	2048	Same size as C memory
C1 memory	2048	New; same size as C memory
D0 memory	2048	Same size as D memory
D1 memory	2048	New; same size as D memory

5.4 Analog Blocks

The analog block hardware registers on Ezairo 7100 have been restructured compared to those on Ezairo 5900 and Ezairo 6200, and the CFX macros that support the analog blocks have been updated accordingly.

The typical set of macros used for analog block configuration in an Ezairo 5900 or Ezairo 6200 application is given below:

```
// Configure the ADCs
Sys_Set_ADC_CFG(ADC_CFG_VAL | SYS_CALC_SF)

// Configure the decimation filter gain factors. In addition to the gain
// values defined in app.cfi, these macros use the parameter SYS_CALC_SF
// which is calculated above by the macro Sys_Calc_Sample_Freq_Val().
Sys_Set_Decimation_Gain01(SYS_CALC_SF, CH0_DF_GAIN, CH1_DF_GAIN)
Sys_Set_Decimation_Gain23(SYS_CALC_SF, CH2_DF_GAIN, CH3_DF_GAIN)

// Configure the input gain
Sys_Set_IN_GAIN_CTRL(IN_GAIN_CTRL_VAL)

// Configure the analog input stage
Sys_Set_INPUT_CTRL(INPUT_CTRL_VAL)
```

The equivalent set of macros for Ezairo 7100 is as follows:

```
// Configure the sampling frequency
Sys_Set_ADC_SAMPLE_FREQ_CFG(ADC_SAMPLE_FREQ_CFG | SYS_CALC_SF)

// Configure the decimation filter gain factors. In addition to the gain
// value defined in app.cfi, this macro uses the parameter SYS_CALC_SF
// which is calculated above by the macro Sys_Calc_Sample_Freq_Val(). The
// output is the define, SYS_CALC_GF, which is the rounded value for the
// desired gain factor and should be used for the ADC_GF field of
// A_ADC_DEC_CTRL_*.
Sys_Calc_Gain_Factor_Val(SYS_CALC_SF, DF_UNITY_GAIN)

// Configure ADC0
Sys_Set_ADC_CTRL(0, ADC0_CTRL_VAL)

// Configure IDRX0 and IDRX1
Sys_Set_IDRX_CFG(0, IDRX_CFG_VAL)
Sys_Set_IDRX_CFG(1, IDRX_CFG_VAL)

// Configure the decimation filter for ADC0
Sys_Set_ADC_DEC_CTRL(0, ADC_DEC_CTRL_0_VAL | SYS_CALC_GF)
```

5.5 CLOCKING

You can simply switch to using the voltage controlled oscillator when porting your Ezairo 5900 or Ezairo 6200 application. You can also take advantage of more advanced clocking features on Ezairo 7100. See the *Hardware Reference Manual for Ezairo 7100* for more information.

5.6 FIFO CONTROLLER INPUT AND OUTPUT BLOCKS

Ezairo 7100 supports eight configurable FIFO interrupts. For each FIFO interrupt, its source can be any of the 32 FIFOs. We recommend that you explicitly configure each enabled FIFO interrupt. If you do not configure the FIFO interrupts, they assume the default settings as shown in Table 12.

Table 12. Default FIFO Interrupt Settings

FIFO Interrupt	Default FIFO Source
FIFO 0	FIFO A0_0
FIFO 1	FIFO A0_1
FIFO 2	FIFO A1_0
FIFO 3	FIFO A1_1
FIFO 4	FIFO B0_0
FIFO 5	FIFO B0_1
FIFO 6	FIFO B1_0
FIFO 7	FIFO B1_1

With Ezairo 7100, the IO block size is limited to 1024 words regardless of whether or not smart FIFO accesses are used. With Ezairo 5900 or Ezairo 6200, the IO block size might be limited to 512 words if smart FIFO accesses are used.

With Ezairo 7100, the maximum FIFO size is 2048. With Ezairo 5900 or Ezairo 6200, the maximum FIFO size is 2016.

5.6.1 FIFO and IOC Macros

When you use the firmware macro `Sys_FIFO_Configure` to configure a FIFO, note that valid FIFO names have changed from `FIFO_0`, `FIFO_1`, etc. for Ezairo 5900 and Ezairo 6200 to `FIFO_A0_0`, `FIFO_A1_0`, etc. for Ezairo 7100.

We recommend using firmware macro `Sys_FIFO_CFXIntConfig` to configure a CFX FIFO interrupt, and `Sys_FIFO_HEARIntConfig` to configure a HEAR FIFO interrupt.

IOC macros have also been updated. The functionality of the macro `Sys_IOC_InputConfig` for Ezairo 5900 and Ezairo 6200 is split into two macros for Ezairo 7100: `Sys_IOC_ADCInputConfig` and `Sys_IOC_PCMInputConfig`. The functionality of the macro `Sys_IOC_FIFOConfig` for Ezairo 5900 and Ezairo 6200 is split into two macros for Ezairo 7100: `Sys_IOC_FIFOShiftConfig` and `Sys_IOC_FIFOAccessConfig`.

5.7 HEAR Library Migration

For Ezairo 5900 or Ezairo 6200 in EDK 5.2, you have the option to use either the new HEAR library (recommended), or the old HEAR library (version 1.x). In Ezairo 7100 EDK, you must use the new HEAR library.

If you are porting an Ezairo 5900 or Ezairo 6200 project that still uses the old HEAR library (version 1.x), you need to take an extra step to migrate to the new HEAR library, preferably before you port the entire project to Ezairo 7100. To migrate the HEAR library, refer to the “Library Migration” subsection in the “Modules” section in the *HEAR Configurable Accelerator Reference Manual* in EDK 5.2 for a concise list of major differences between the two HEAR library versions, and refer to the “Modules” and “Module Reference” sections for detailed information.

5.8 Digital Input/Output (DIO) Interface

The Ezairo 7100 system’s DIO interface is an extension of the GPIO interface for Ezairo 5900 and Ezairo 6200. Thirty DIO pads can be configured for a variety of digital output modes, for a general-purpose input mode, or as an LSAD input.

5.8.1 DIO Macros

A set of DIO macros have been added to support DIO functionality, as described in Section 5.8, “Digital Input/ Output (DIO) Interface” on page 44. If you are porting an Ezairo 5900 or Ezairo 6200 application that uses external interfaces, you might need to use the DIO macros listed in Table 13 to configure the interfaces. For more information about these macros, and for a complete list of DIO macros, see the *Firmware Reference Manual for Ezairo 7100*.

Table 13. Useful DIO Macros for Porting

Macro	Description
Sys_DIO_Config	Configure the specified digital I/O
Sys_DIO_Config_PCM	Configure four DIOs as the CFX PCM interface
Sys_DIO_Config_SPI	Configure four DIOs as the CFX SPI interface

Table 13. Useful DIO Macros for Porting (Continued)

Macro	Description
Sys_DIO_Config_UART	Configure two DIOs as the CFX UART interface
Sys_DIO_Get_Mode	Get the DIO mode (DIO or GPIO) and store it in the specified accumulator register

5.8.2 GPIO Macros

Table 14 lists all the GPIO macros, and identifies the macros that are new or updated for Ezairo 7100.

Table 14. GPIO Macros

Macro	New or Updated	Description
Sys_GPIO_Direction		Set the GPIO directions
Sys_GPIO_Get_Data		Get the GPIO input and store it in the specified accumulator register
Sys_GPIO_Get_Data_1	X	Get the GPIO input for the VDDO 1 supported DIOs and store it in the specified register
Sys_GPIO_Get_Data_2	X	Get the GPIO input for the VDDO 2 supported DIOs and store it in the specified register
Sys_GPIO_Get_Data_3	X	Get the GPIO input for the VDDO 3 supported DIOs and store it in the specified register
Sys_GPIO_Get_IntStatus	X	Get the current status for the DIO interrupts; clears the current status
Sys_GPIO_Get_IntStatusCM3	X	Get the current status for the ARM Cortex-M3 core DIO interrupts; clears the current status
Sys_GPIO_IntConfig	X	Configure a CFX GPIO interrupt source
Sys_GPIO_IntConfig_CM3	X	Configure an ARM Cortex-M3 core GPIO interrupt source
Sys_GPIO_Set_Data		Set the GPIO output to the specified value
Sys_GPIO_Set_Data_1	X	Set the GPIO output for the VDDO 1 supported DIOs to the specified value
Sys_GPIO_Set_Data_2	X	Set the GPIO output for the VDDO 2 supported DIOs to the specified value
Sys_GPIO_Set_Data_3	X	Set the GPIO output for the VDDO 3 supported DIOs to the specified value
Sys_GPIO_Set_High		Set the specified GPIO output value to high
Sys_GPIO_Set_Low		Set the specified GPIO output value to low
Sys_GPIO_Toggle		Toggle the specified GPIO output pin
Sys_GPIO_Toggle_Bits		Toggle the specified GPIO output pins
Sys_GPIO_Toggle_Bits_1	X	Toggle the specified GPIO output pins supported by VDDO 1
Sys_GPIO_Toggle_Bits_2	X	Toggle the specified GPIO output pins supported by VDDO 2
Sys_GPIO_Toggle_Bits_3	X	Toggle the specified GPIO output pins supported by VDDO 3

5.9 EEPROM LIBRARY

Ezairo 5900 and Ezairo 6200 support 16-bit addressed EEPROM devices. Ezairo 7100 has added support for 24-bit addressed devices and configurable page size. The EEPROM library has two new macros to support the new

functionality, as listed in Table 15. In addition, a new EEPROM library function, `EELIB_AutoSet_AddressMode`, finds the address mode of the EEPROM attached to the SPI interface, and sets the address mode accordingly. This is useful when the required address mode for the attached EEPROM is unknown.

Table 15. New EEPROM Library Macros

Macro	Description
EELIB_Set_AddressMode	Set the address mode for the EEPROM library
EELIB_Set_PageSize	Set the page size for the EEPROM library

5.10 PROGRAM ROM

The program ROM for Ezairo 7100 has been updated to reflect the changes of the EEPROM file system and includes the new EEPROM function `EELIB_AutoSet_AddressMode`. Otherwise, it is the same as the program ROM for Ezairo 5900 and 6200.

CHAPTER A
Glossary

The following abbreviations and terms are used in this manual:

algorithm partitioning
: Separating your algorithm to assign different parts of it to run on multiple processors.

CFX Digital Signal Processor
: the Ezairo 7100 processing core that is used to configure the system and coordinate the flow of signal data progressing through the system

data sink
: Something that reads data from a FIFO.

data source
: Something that writes data into a FIFO.

delay slot
: A technique used to reduce the effect of pipeline breaks by executing the instruction (in the delay slot) that is after a control flow instruction before performing the control flow action.

DIO
: digital input/output

FIFO
: A section of memory that is treated as a circular buffer where components that use this memory use pointers to handle data in a first-in, first-out manner. A *hardware FIFO* is a FIFO instance where its registers are controlled by the IOC. A *software FIFO* is a FIFO instance where its registers are controlled by an application on the CFX or microcode configuration on the HEAR.

FIFO, hardware
: FIFOs that are connected to a physical interface through the IOC.

FIFO, software
: FIFOs that are controlled by the software on the CFX and HEAR.

function chain
: A sequence of microcode functions that are executed on the HEAR Configurable Accelerator in response to a command from the CFX or an event from the FIFO controller.

GPIO
: general-purpose input/output

HEAR Configurable Accelerator
: the Ezairo 7100 core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions; designed to work with the CFX to reduce the signal processing load on the CFX

I^2C
: Industry standardized Inter-IC Communication protocol.

I^2S
: Industry standardized Inter-IC Sound protocol

IOC
: input/output controller

ISR
: interrupt service routine

LSAD
: low-speed analog-to-digital

NVM
: non-volatile memory

PCM
: pulse code modulation

SPI
: serial peripheral interface

time-slicing
: A technique for partitioning your algorithm over the time dimension to free up clock cycles for other more critical computations.

UART
: universal asynchronous receiver-transmitter

WDRC
: Wide Dynamic Range Compression

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo and SignaKlara are either trademarks or registered trademarks of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20797-010